Федеральное агентство по рыболовству
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Астраханский государственный технический университет»
Система менеджмента качества в области образования, воспитания, науки и инноваций сертифицирована DQS по международному стандарту ISO 9001:2015

Институт Информационных технологий и коммуникаций

Кафедра Автоматизированные системы обработки информации и управления

Параллельное программирование
Методические указания к лабораторным работам

Для обучающихся (очная форма обучения)
по направлению подготовки

09.03.04 Программная инженерия
Профиль
Разработка программно – информационных систем

АСТРАХАНЬ – 2023

Автор(ы): доцент, к.т.н. Белов С.В.

Рецензент: профессор, д.т.н., зав. кафедрой АСОИУ Хоменко Т.В.

Методические указания к проведению занятий по дисциплине «Параллельное программирование» утверждены на заседании кафедры «Автоматизированные системы обработки информации и управления» «31» августа 2023 г., протокол № 1.

СОДЕРЖАНИЕ

ЛАБОРАТОРНАЯ РАБОТА №1 ЗНАКОМСТВО С ПОТОКАМИ В C#.....2
ЛАБОРАТОРНАЯ РАБОТА №2 ПРОСТЕЙШАЯ ОБРАБОТКА ЭЛЕМЕНТОВ ВЕКТОРА.....4
ЛАБОРАТОРНАЯ РАБОТА №3 ПРОСТЫЕ ЧИСЛА.....9
ЛАБОРАТОРНАЯ РАБОТА №4 СИНХРОНИЗАЦИЯ ДОСТУПА К БУФЕРУ….15
ЛАБОРАТОРНАЯ РАБОТА №5 СИНХРОНИЗАЦИЯ ПРИОРИТЕТНОГО ДОСТУПА К МНОГОЭЛЕМЕНТНОМУ БУФЕРУ.....23
ЛАБОРАТОРНАЯ РАБОТА №6. КЛЕТОЧНАЯ МОДЕЛЬ «ИГРА ЖИЗНЬ».....30
Лабораторные работы №7. Процессы в операционной системе UNIX.....33
Лабораторная работа №8. Организация взаимодействия процессов через pipe и FIFO в UNIX.....40
Лабораторные работы №9. Средства System V IPC. Организация работы с разделяемой памятью в UNIX. Понятие нитей исполнения (thread'ов).....47
Лабораторная работа №10. Семафоры в UNIX как средство синхронизации процессов.....55
Лабораторная работа № 11. Очереди сообщений в UNIX.....58
Лабораторные работы №12. Организация ввода-вывода в UNIX. Файлы устройств. Аппарат прерываний. Сигналы в UNIX.....63

ЛАБОРАТОРНАЯ РАБОТА №1 ЗНАКОМСТВО С ПОТОКАМИ В C#

Цель работы

Получить общие представления о потоках, о способах их создания и использования.

Основные теоретические положения

Лабораторная работа выполняется на основе материала, разобранного на практическом занятии по теме «Работа с потоками в C#».

Задание предполагает выполнение в консольном приложении и на компьютере с многоядерным процессором.

Порядок выполнения работы

1. Перед началом выполнения определите, сколько ядер на вашем процессоре (далее будем предполагать что *N*).
2. Провести некоторые исследования, как может повлиять использование потоков на скорость выполнения программы, а также сравним между собой, как влияет на скорость выполнения программы использование различных способов создания потоков.
3. Результаты будет необходимо представить в виде таблицы 1. Таблица 1

Сложность алгоритма	последовательная обработка		многопоточная, инициализация с помощью лямбдавыражений		многопоточная, инициализация с помощью статического метода		многопоточная, с помощью пула потоков	
	время работы	порядок вывода	время работы	порядок вывода	время работы	порядок вывода	время работы	порядок вывода
(1)	*(2)*	*(3)*	*(4)*	*(5)*	*(6)*	*(7)*	*(8)*	*(9)*
1 блок								
2 блока								
…								
N блоков								
N+1 блок								

Задание

Пусть имеется некоторый фрагмент кода:

```
double x = 12345.6789;
     for (int i = 0; i < 10000; i++)
      for (int j = 0; j < 10000; j++)
      {
          x = Math.Sqrt(x);
          x = x + 0.000000001;
         x = Math.Pow(x, 2);
      }
Console.WriteLine("A: " + x.ToString());
```

1. Определите, сколько времени выполняется данный фрагмент кода на вашем компьютере. Для этого можно, например, в начале программы вставить такой код:

```
DateTime t1 = DateTime.Now;
```

а в конце следующий, определяющий время выполнения и его вывод на экран:

```
DateTime t2 = DateTime.Now;
Console.WriteLine(t2-t1);
```

2. Определите, сколько времени будет выполняться при последовательной обработке программа, содержащая два таких фрагмента следующих друг за другом; три фрагмента и т.д., до ***N+1***, где ***N*** – количество ядер процессора вашего компьютера. Запишите результаты в столбец *(2)* таблицы.

3. Для пункта 2 посмотрите, в какой последовательности результаты выполнения фрагментов кода выводятся на экран (для этого во втором блоке в строке вывода вместо «*A:*» запишите «*B:*», в третьем – «*C:*» и т.д.).
Запишите результаты в столбец *(3)* таблицы (например, ABC).

4. Определите, сколько времени будет выполняться при многопоточной обработке программа, выполняющая два фрагмента кода (см. выше) в разных потоках; три фрагмента в разных потоках и т.д., до ***N+1***. Потоки при этом создавайте с использованием лямбда-выражений. Не забудьте заблокировать выполнение основного процесса до завершения работы всех потоков. Запишите результаты в столбец *(4)* таблицы.

5. Для пункта 4 посмотрите, в какой последовательности результаты выполнения фрагментов кода выводятся на экран. Запишите результаты в столбец *(5)*.

6. Выполните пункт 4 снова, при этом сам фрагмент кода оформите в виде статической процедуры и при создании потоков задавайте указанную

процедуру. Процедура должна содержать один параметр: литеру (букву), обозначающую поток (для вывода результата на экран). Запишите результаты в столбец *(6)* таблицы.

7. Для пункта 6 посмотрите, в какой последовательности результаты выполнения фрагментов кода выводятся на экран. Запишите результаты в столбец *(7)*.

8. Выполните пункт 4 снова, при этом для создания потоков используйте пул. Запишите результаты в столбец *(8)* таблицы.

9. Для пункта 8 посмотрите, в какой последовательности результаты выполнения фрагментов кода выводятся на экран. Запишите результаты в столбец *(9)*.

10. Для пункта 6 исследуйте, как влияет приоритет потоков на порядок их выполнения. Присвойте каждому потоку разный приоритет и сравните результаты, полученные при количестве потоков меньше количества ядер процессора, при количестве потоков равному количеству ядер, при количестве потоков больше количества ядер. Сделайте вывод: в каких ситуация приоритет повлиял на порядок вывода результата на экран, в каких нет.

Требования к отчету

Отчет по лабораторной работе должен состоять из:

1. информацию о студенте – ФИО, группа;
2. цель работы;
3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);
4. заполненную таблицу;
5. выводы по п. 10 задания с указанием установленных потокам приоритетов и порядка завершения их выполнения;
6. исходный код для максимального числа блоков/потоков по пп. 2, 4, 6, 8 и 10 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. Что такое потоки? Для чего они нужны? Какими свойствами обладают?
3. Что такое Лямбда-выражение?
4. Как создать поток с использованием лямбда-выражений?

ЛАБОРАТОРНАЯ РАБОТА №2 ПРОСТЕЙШАЯ ОБРАБОТКА ЭЛЕМЕНТОВ ВЕКТОРА

Цель работы

Получить представления об эффективности применения параллельного программирования для простейшей обработки элементов вектора.

Основные теоретические положения

В работе исследуется эффективность распараллеливания независимой обработки элементов вектора. В качестве обработки можно выбрать математическое преобразование элементов вектора, например – возведение в степень:

```
for(int i=0; i<a.Length; i++)
      b[i] = Math.Pow(a[i], 1.789);
```

В функции потока будет необходимо предусмотреть возможность разбиения диапазона *0..N-1* на число потоков *M* (рис. 1). При запуске потока в качестве аргумента должен передаваться либо "индекс потока", определяющий область массива, который обрабатывается в данном потоке, либо начальный и конечный индексы массива.

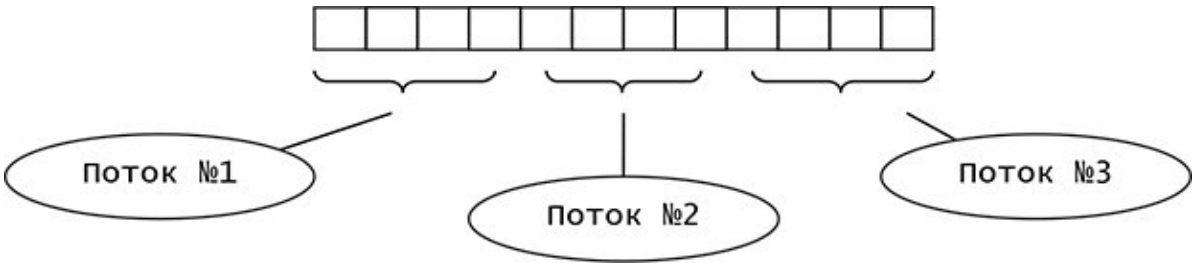


Рисунок 1 – Разбиение диапазона на потоки

Многопоточное выполнение будет параллельным при наличии в вычислительной системе нескольких процессоров (ядер процессора). Число процессоров можно узнать с помощью свойства:

```
System.Environment.ProccessorCount;
```

Параллельное выполнение вычислений также можно реализовать с помощью классов библиотеки ***TPL*** (*Task Parallel Library*). Классы библиотеки располагаются в пространстве имен *System.Threading.Tasks*. Параллельное вычисление операций над элементами цикла выполняется с помощью метода *Parallel.For*:

```
Parallel.For(0, a.Length, i =>
      {  b[i] = Math.Pow(a[i], 1.789);  });
```

Для анализа производительности последовательного и параллельного выполнения можно использовать переменные типа *DateTime*. Также можно использовать объект *Stopwatch* пространства *System.Diagnostics*:

```
      Stopwatch sw = new Stopwatch();
sw.Start();
      // Вызов_вычислительной_процедуры;
      sw.Stop();
      TimeSpan ts = sw.Elapsed;
      Console.WriteLine("Total time: {0}",
ts.TotalMilliseconds);
```

При оценке производительности необходимо учесть, что время выполнения алгоритма зависит от множества параметров. Поэтому желательно оценивать среднее время выполнения при нескольких прогонах алгоритма.

Эффективность параллельного алгоритма существенно зависит от элементов массива, числа потоков, сложности математической функции и т.д. Следует учитывать, что при малом объеме элементов массива, накладные расходы, связанные с организацией многопоточной обработки, превышают выигрыш от параллельности обработки. При последовательном выполнении примитивной циклической обработки быстродействие достигается за счет оптимального использования кэш-памяти.

Выполняя анализ зависимости быстродействия от числа потоков, следует учитывать число ядер процессора. Увеличение числа потоков сверх возможностей вычислительной системы приводит к конкуренции потоков и ухудшению быстродействия.

Усложнение обработки элементов массива предлагается реализовать с помощью внутреннего цикла. Например,

```
      for(int i=0; i<a.Length; i++)
      {
            // Обработка i-элемента
for (int j=0; j < K; j++)
                  b[i] += Math.Pow(a[i], 1.789);
}
```

где *K* – параметр "сложности". Увеличивая параметр *K*, наблюдаем повышение эффективности параллельной обработки при меньшем объеме массива чисел.

В рассмотренных вариантах обработки вычислительная нагрузка на каждой итерации относительно одинакова. В ситуациях, когда вычислительная нагрузка

зависит от индекса элемента, разделение массива по равным диапазонам может быть не эффективно. Рассмотрим следующий вариант обработки:

```
        for (int i=0; i<a.Length; i++)
        {
                // Обработка i-элемента
for (int j=0; j <i; j++)
                    b[i] += Math.Pow(a[i], 1.789);
}
```

Здесь вычислительная нагрузка при обработке *i*-элемента зависит от индекса *i*. Обработка начальных элементов массива занимает меньшее время по сравнению с обработкой последних элементов. Разделение данных по диапазону приводит к несбалансированной загрузке потоков и снижению эффективности распараллеливания (рис. 2).

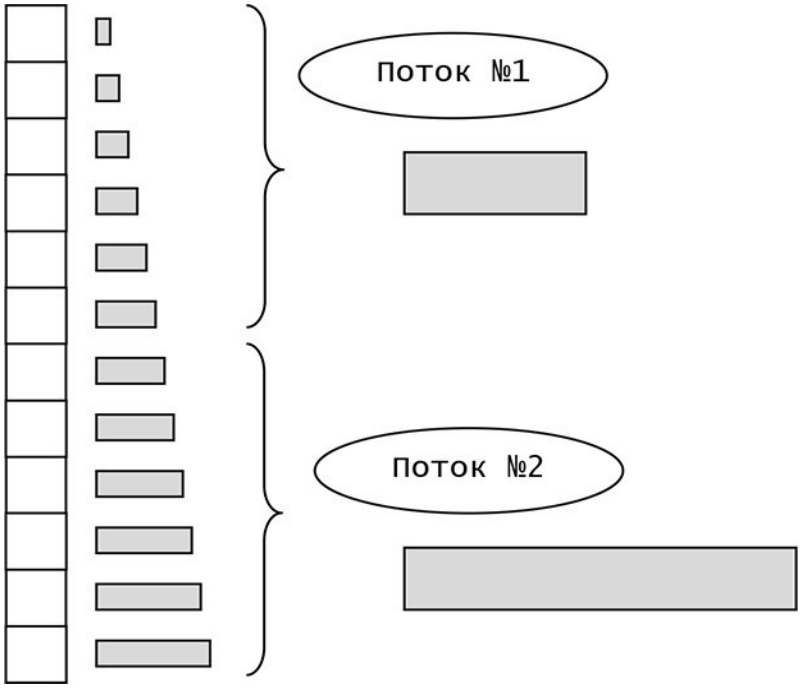


Рисунок 2 – Несбалансированная загрузка потоков

Одним из подходов к выравниванию загрузки потоков является применение *круговой декомпозиции*. Так, например, в случае двух потоков получаем такую схему: первый поток обрабатывает все четные элементы, второй поток обрабатывает все нечетные элементы.

Порядок выполнения работы

1. Изучить основные теоретические положения, и выполнить предоставленные задания.

2. По результатам задания 3 и 4 заполнить таблицу 2.

Таблица 2

Кол-во элементов	Последовательная обработка		2 потока		…	
	простая	усложнен.	простая	усложнен.	простая	усложнен.
(1)	*(2)*	*(3)*	*(4)*	*(5)*	*(8)*	*(9)*
10						
100						
…						

3. По результатам задания 5 и 6 заполнить таблицу 3.

Таблица 3

Кол-во элементов	2 потока		3 потока		4 потока		…	
	по диап.	по кругу	по диап.	по кругу	по диап.	по кругу	по диап.	по кругу
(1)	*(2)*	*(3)*	*(4)*	*(5)*	*(6)*	*(7)*	*(8)*	*(9)*
10								
100								
…								

Задание

1. Реализуйте последовательную обработку элементов вектора, например: возведение элемента вектора в какую-либо степень. Число элементов вектора задается параметром N.
2. Реализуйте многопоточную обработку элементов вектора, используя разделение вектора на равное число элементов. Число потоков задается параметром M.
3. Выполните анализ эффективности многопоточной обработки при разных параметрах N (10, 100, 1000, 100000) и M (2, 3, 4, 5, 10). Результаты представьте в табличной форме.
4. Выполните анализ эффективности при усложнении обработки каждого элемента вектора.
5. Исследуйте эффективность разделения по диапазону при неравномерной вычислительной сложности обработки элементов вектора.
6. Исследуйте эффективность параллелизма при круговом разделении элементов вектора. Сравните с эффективностью разделения по диапазону.

Требования к отчету Отчет должен содержать:

1. информацию о студенте – ФИО, группа;
2. цель работы;
3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);
4. таблицу 1 по итогам выполнения пп. 3 и 4 задания;
5. таблицу 2 по итогам выполнения пп. 5 и 6 задания;
6. исходный код, полученный при выполнении задания по пунктам 1, 2 и 6 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. Что такое вектор? Приведите пример применения.
3. Как реализовать обработку элементов вектора с применение параллельного программирования?

ЛАБОРАТОРНАЯ РАБОТА №3 ПРОСТЫЕ ЧИСЛА

Цель работы

Провести анализ эффективности применения различных параллельных алгоритмов к решению одной и той же задачи – поиска простых чисел.

Основные теоретические положения

Для выполнения данной лабораторной работы рассмотрим несколько алгоритмов поиска простых чисел.

Последовательный алгоритм "Решето Эратосфена"

Алгоритм заключается в последовательном переборе уже известных простых чисел, начиная с двойки, и проверке разложимости всех чисел диапазона (m, n] на найденное простое число m. На первом шаге выбирается число m=2, проверяется разложимость чисел диапазона (2, n] на 2-ку. Числа, которые делятся на двойку, помечаются как составные и не участвуют в дальнейшем анализе. Следующим непомеченным (простым) числом будет m=3, и так далее (рис. 3).

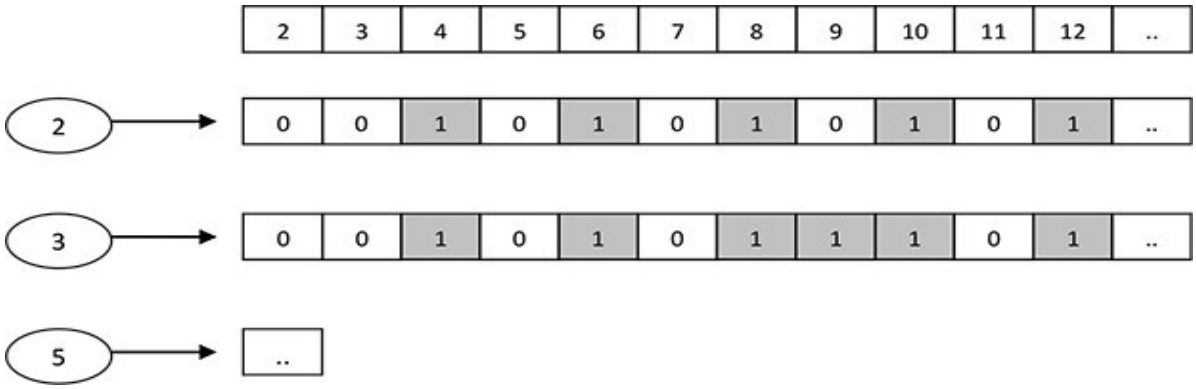


Рисунок 3 – Алгоритм «Решето Эратосфена»

При этом достаточно проверить разложимость чисел на простые числа в интервале $[2..\sqrt{n}]$. Например, в интервале от 2 до 20 проверяем все числа на разложимость 2, 3. Составных чисел, которые делятся только на пятерку, в этом диапазоне нет.

Модифицированный последовательный алгоритм поиска

В последовательном алгоритме "базовые" простые числа определяются поочередно. После тройки следует пятерка, так как четверка исключается при обработке двойки. Последовательность нахождения простых чисел затрудняет распараллеливание алгоритма. В модифицированном алгоритме выделяются два этапа:

1-ый этап: поиск простых чисел в интервале от $2 \ldots \sqrt{n}$ с помощью классического метода решета Эратосфена (базовые простые числа).

2-ой этап: поиск простых чисел в интервале от $\sqrt{n} \ldots n$, в проверке участвуют базовые простые числа, выявленные на первом этапе (рис. 4).

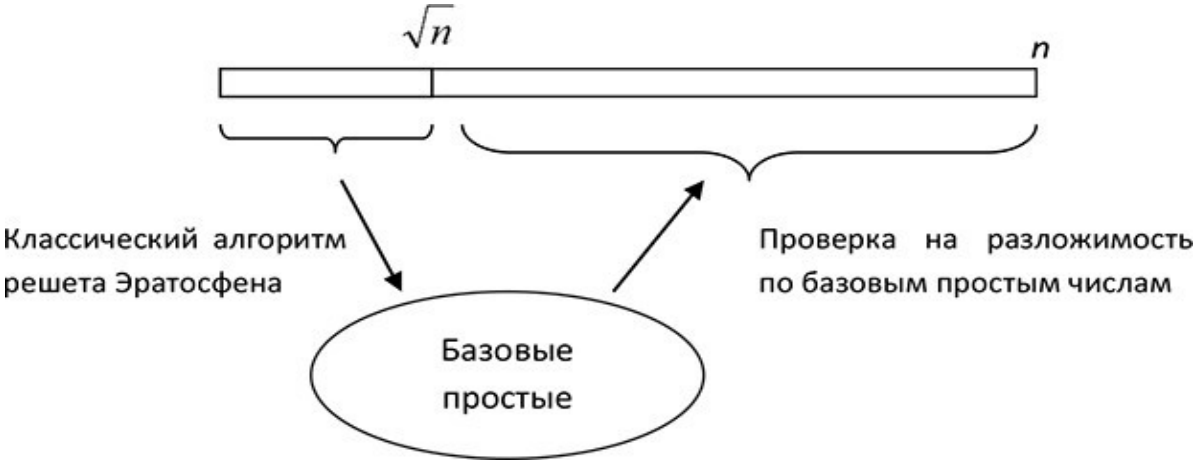


Рисунок 4 – Модифицированный последовательный алгоритм поиска

На первом этапе алгоритма выполняется сравнительно небольшой объем работы, поэтому нецелесообразно распараллеливать этот этап. На втором этапе проверяются уже найденные базовые простые числа. Параллельные алгоритмы разрабатываются для второго этапа.

Параллельный алгоритм №1: декомпозиция по данным (рис. 5)

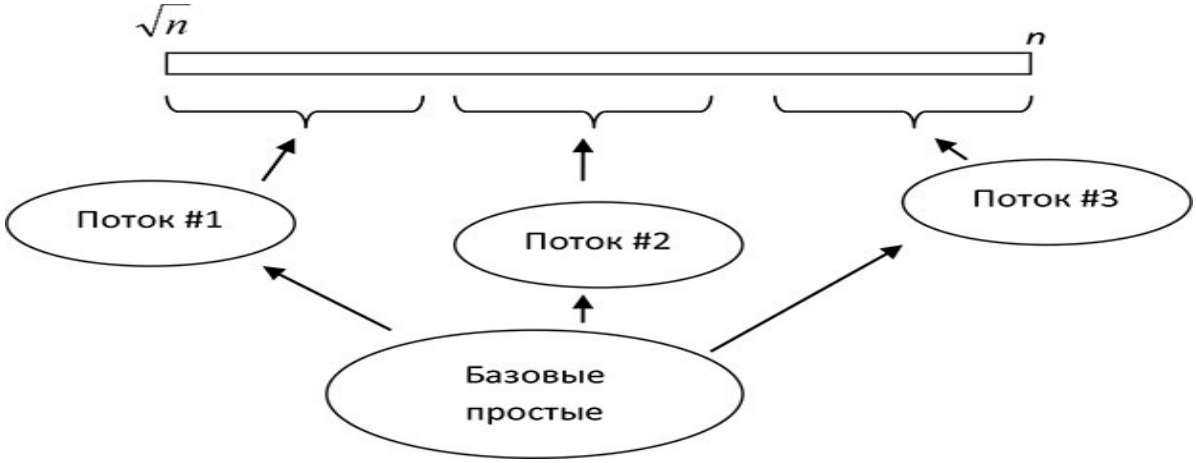
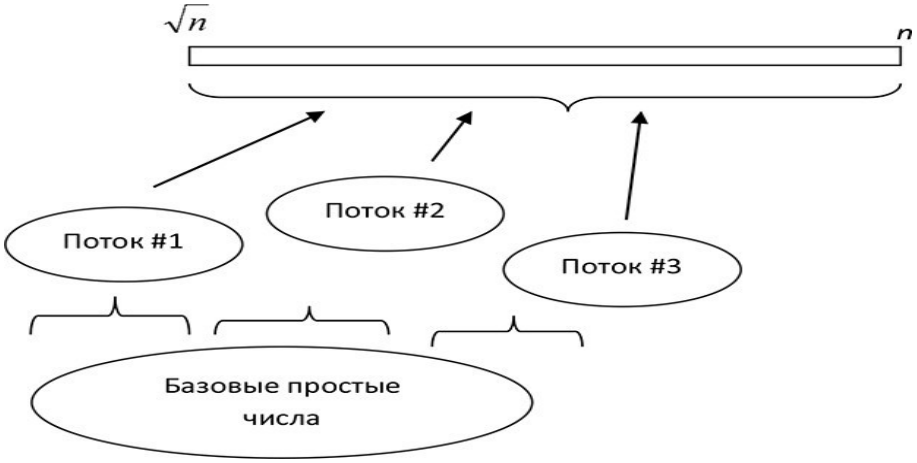


Рисунок 5 – Параллельный алгоритм №1

Идея распараллеливания заключается в разбиении диапазона $\sqrt{n} \ldots n$ на равные части. Каждый поток обрабатывает свою часть чисел, проверяя на разложимость по каждому базовому простому числу.

Параллельный алгоритм №2: декомпозиция набора простых чисел

В этом алгоритме разделяются базовые простые числа. Каждый поток работает с $\sqrt{n} \ldots n$ (рис. 6).



ограниченным набором простых чисел и проверяет весь диапазон

Рисунок 6 – Параллельный алгоритм №2

Параллельный алгоритм №3: применение пула потоков

Применение пула потоков позволяет автоматизировать обработку независимых рабочих элементов. В качестве рабочих элементов предлагается использовать проверку всех чисел диапазона от $\sqrt{n} \ldots n$ на разложимость по одному базовому простому числу (рис. 7).

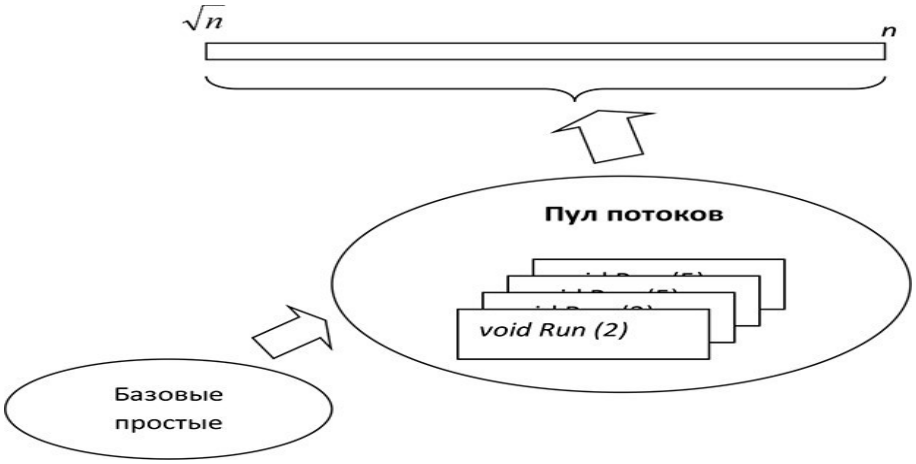


Рисунок 7 – Параллельный алгоритм №3

Для применения пула потоков необходимо загрузить рабочие элементы вместе с необходимыми параметрами в очередь пула потоков:

```
for(int i=0; i<basePrime.Length; i++)
  {
      ThreadPool.QueueUserWorkItem(Run, basePrime[i]);      }
```

Run – метод обработки всех чисел диапазона $\sqrt{n} \ldots n$ на разложимость простому числу *basePrime*[*i*].

Выполнение рабочих элементов осуществляется автоматически после добавления в пул потоков. Не существует встроенного механизма ожидания завершения рабочих элементов, добавленных в пул потоков. Поэтому вызывающий поток (метод *Main*) должен контролировать завершение либо с помощью средств синхронизации (например, сигнальных сообщений), либо с помощью общих переменных и цикла ожидания в методе *Main*.

Применение сигнальных сообщений может быть реализовано следующим образом:

```
static void Main()
{
  // Поиск базовых простых
  ..   int[] basePrime = ..
```

```
  // Объявляем массив сигнальных сообщений   ManualResetEvent []
events =
    new ManualResetEvent [basePrime.Length];

   // Добавляем в пул рабочие элементы с параметрами   for(int
i=0; i<basePrime.Length; i++)
  {
    events[i] = new ManualResetEvent(false);
    ThreadPool.QueueUserWorkItem(Run,      new
object[] {basePrime[i], events[i]})
  }
  // Дожидаемся завершения
  WaitHandle.WaitAll(events);

  // Выводим результаты
  .. }
static void F(object o)
{
  int prime = (int)((object[])o)[0];
  ManualResetEvent ev = ((object[])o)[1] as ManualResetEvent;   //
Обработка чисел на разложимость простому числу prime
  ..   ev.Set();
}
```

Параллельный алгоритм №4: последовательный перебор простых чисел

Идея алгоритма заключается в последовательном переборе базовых простых чисел разными потоками. Каждый поток осуществляет проверку всего диапазона на разложимость по определенному простому числу. После обработки первого простого числа поток не завершает работу, а обращается за следующим необработанным простым числом (рис. 8).

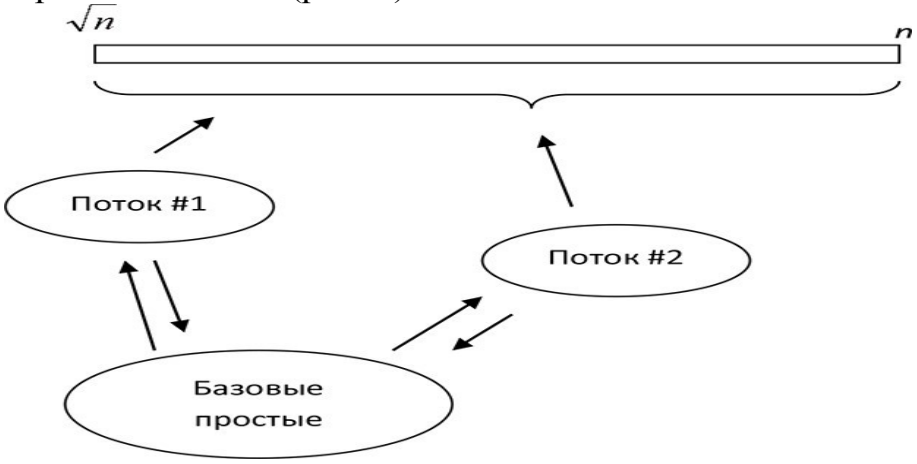


Рисунок 8 – Параллельный алгоритм №4

Для получения текущего простого числа поток выполняет несколько операторов:

```
while(true)
{
  if (current_index >= basePrime.Length) break;   current_prime
= basePrime[current_index];   current_index++;
  // Обработка текущего простого числа
  ..
}
```

В этой реализации существует *разделяемый ресурс* – массив простых чисел. При одновременном доступе к ресурсу возникает проблема *гонки данных*. Следствием этой проблемы являются: лишняя обработка, если несколько потоков одновременно получают одно и то же число; пропущенная задача – потоки, получив одно число, последовательно увеличивают текущий индекс; исключение "Выход за пределы массива", когда один поток успешно прошел проверку текущего индекса, но перед обращением к элементу массива, другой поток увеличивает текущий индекс.

Для устранения проблем с совместным доступом необходимо использовать *средства синхронизации* (критические секции, атомарные операторы, потокобезопасные коллекции).

Критическая секция позволяет ограничить доступ к блоку кода, если один поток уже начал выполнять операторы секции: `lock (sync_obj)`

```
{
  критическая_секция
}
```

где *sync_obj* – объект синхронизации, идентифицирующий критическую секцию (например, строковая константа).

Порядок выполнения работы

1. Изучить основные теоретические положения, и выполнить предоставленные задания;
2. По результатам задания заполнить таблицу (форма таблицы – на ваше усмотрение).

Задание

1. Реализуйте последовательный и параллельные алгоритмы поиска простых чисел.
2. Выполните анализ быстродействия алгоритмов при разном объеме данных, разном числе потоков.
3. Рассчитайте ускорение и эффективность выполнения алгоритмов.

4. Сделайте выводы о целесообразности применения параллельных алгоритмов и необходимости использования синхронизации.

Требования к отчету

Отчет должен содержать:

1. информацию о студенте – ФИО, группа;
2. цель работы;
3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);
4. результаты в табличной форме по итогам выполнения п. 2 задания (форма таблицы – на ваше усмотрение);
5. результаты по итогам выполнения п. 3 задания;
6. выводы по п.4 задания;
7. исходный код, полученный при выполнении п. 1 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. Что такое простые числа?
3. В чем заключается алгоритм "Решето Эратосфена"?
4. В чем заключается Модифицированный последовательный алгоритм поиска простых чисел?
5. В чем заключается Параллельный алгоритм №1: декомпозиция по данным?
6. В чем заключается Параллельный алгоритм №2: декомпозиция набора простых чисел?
7. В чем заключается Параллельный алгоритм №3: применение пула потоков?
8. В чем заключается Параллельный алгоритм №4: последовательный перебор простых чисел?

ЛАБОРАТОРНАЯ РАБОТА №4 СИНХРОНИЗАЦИЯ ДОСТУПА К БУФЕРУ

Цель работы

Реализовать работу нескольких потоков с одноэлементным буфером с использованием различных средств синхронизации.

Основные теоретические положения

В случае одноэлементного буфера достаточно использовать флаг типа *bool* для контроля состояния буфера. Читатели обращаются к буферу, только если он свободен:

```
// Работа читателя   while (!
finish)
{
    if (!bEmpty)
    {
        MyMessages.Add(buffer);
bEmpty = true;
     }
}
```

Писатели обращаются к буферу, только если он пуст:

```
// Работа писателя  while(i < n)
{
    if (bEmpty)
    {
        buffer = MyMessages[i++];
         bEmpty = false;
    }
}
```

Писатели работают, пока не запишут все свои сообщения. По окончании работы писателей основной поток может изменить статус переменной *finish*, который является признаком окончания работы читателей.

```
static void Main()
{
    // Запускаем читателей и писателей
    ..
    // Ожидаем завершения работы писателей
for(int i=0; i< writers.Length; i++)
writers[i].Join();
    // Сигнал о завершении работы для читателей      finish
= true;
          // Ожидаем завершения работы читателей
for(int i=0; i < readers.Length; i++)
readers[i].Join();
}
```

Отсутствие средств синхронизации при обращении к буферу приводит к появлению гонки данных – несколько читателей могут прочитать одно и то же сообщение, прежде чем успеют обновить статус буфера; несколько писателей могут одновременно осуществить запись в буфер. В данной задаче следствием

гонки данных является потеря одних сообщений и дублирование других. Для фиксации проблемы предлагается выводить на экран число повторяющихся и потерянных сообщений.

Самый простой вариант решения проблемы заключается в использовании критической секции (*lock* или *Monitor*).

```
// Работа читателя  while (!
finish)
{
    lock ("read")
    {
        if (!bEmpty)
        {
            MyMessage[i++] = buffer;
bEmpty = true;
        }
    }

}
```

Для писателей существует своя критическая секция:

```
// Работа писателя  while(i <
n)
{
    lock("write")
    {
        if (bEmpty)
        {
            buffer = MyMessage[i++];
bEmpty = false;
        }
    }
}
```

Данная реализация не является оптимальной. Каждый из читателей поочередно входит в критическую секцию и проверяет состояние буфера, в это время другие читатели блокируются, ожидая освобождения секции.

Если буфер свободен, то синхронизация читателей избыточна. Более эффективным является вариант двойной проверки:

```
// Работа читателя  while (!
finish)
{
```

```
    if (!bEmpty)
    {
        lock ("read")
        {
            if (!bEmpty)
            {
                bEmpty = true;
                MyMessage[i++] = buffer;
            }
        }
    }
}
```

Если буфер свободен, то читатели "крутятся" в цикле, проверяя состояние буфера. При этом читатели не блокируются. Как только буфер заполняется, несколько читателей, но не все, успевают войти в первый *if*-блок, прежде чем самый быстрый читатель успеет изменить статус буфера *bEmpty = true*.

Применение сигнальных сообщений позволяет упростить логику синхронизации доступа. Читатели ожидают сигнала о поступлении сообщения, писатели – сигнала об опустошении буфера. Читатель, освобождающий буфер, сигнализирует об опустошении. Писатель, заполняющий буфер, сигнализирует о наполнении буфера. Сообщения с автоматическим сбросом *AutoResetEvent* обладают полезным свойством – при блокировке нескольких потоков на одном и том же объекте *AutoResetEvent* появление сигнала освобождает только один поток, другие потоки остаются заблокированными. Порядок освобождения потоков при поступлении сигнала не известен, но в данной задаче это не существенно.

```
// Работа читателя  void
Reader(object state)
{
    var evFull = state[0] as AutoResetEvent;      var
evEmpty = state[1] as AutoResetEvent;      while(!
finish)
    {
        evFull.WaitOne();
MyMessage.Add(buffer);          evEmpty.Set();
    }
    // Работа писателя  void
Writer(object state)
{
```

```
    var evFull = state[0] as AutoResetEvent;      var
evEmpty = state[1] as AutoResetEvent;      while(i <
n)
    {
        evEmpty.WaitOne();
buffer = MyMessage[i++];
evFull.Set();
    }
}
```

Данный фрагмент приводит к зависанию работы читателей. Писатели закончили работу, а читатели ждут сигнала о наполненности буфера *evFull*. Для разблокировки читателей необходимо сформировать сигналы *evFull.Set()* от писателей при завершении работы или от главного потока. Чтобы отличить ситуацию завершения можно осуществлять проверку статуса *finish* непосредственно после разблокировки.

```
// Рабочий цикл читателей
while(true)  {
    evFull.Wait();
    // Сигнал о завершении работы
if(finish) break;      MyMessage.Add(buffer);
evEmpty.Set();
}
```

Применение семафоров (*Semaphore*, *SemaphoreSlim*) в данной задаче аналогично использованию сигнальных сообщений *AutoResetEvent*. Кроме предложенного варианта обмена сигналами между читателями и писателями, семафоры и сигнальные сообщения могут использоваться в качестве критической секции читателей и писателей. `void Reader(object state)`

```
{
    var semReader = state as SemaphoreSlim;
while(!finish)
    {
        if(!bEmpty)
        {
            semReader.Wait();
if(!bEmpty)
            {
                bEmpty = true;
myMessages.Add(buffer);
            }
```

```
            semReader.Release();
        }
    }
}
void Writer(object state)
{
    var semWriter = state as SemaphoreSlim;
while(i < myMessages.Length)
    {
        if(bEmpty)
{
            semWriter.Wait();
if(bEmpty)                {
                bEmpty = false;
buffer = myMessages[i];
            }
            semWriter.Release();
        }
    }
    }
```

Порядок выполнения работы

1. Изучить основные теоретические положения, и выполнить предоставленные задания.
2. Сделать выводы по итогам выполнения 1-ого задания.
3. По результатам задания 3 заполнить таблицу (форма таблицы – на ваше усмотрение).
4. Сделать выводы по итогам выполнения 4-ого задания.

Задание

Несколько потоков работают с общим одноэлементным буфером. Потоки делятся на «*писателей*», осуществляющих запись сообщений в буфер, и «*читателей*», осуществляющих извлечение сообщений из буфера.

Работу с буфером может осуществлять только один поток. Если буфер свободен, то только один писатель может осуществлять запись в буфер. Если буфер занят, то только один читатель может осуществлять чтение из буфера. После чтения буфер освобождается и доступен для записи.

В качестве буфера используется глобальная переменная, например, типа *string*. Работа приложения заканчивается после того, как все сообщения писателей через общий буфер будут обработаны читателями (рис. 9).

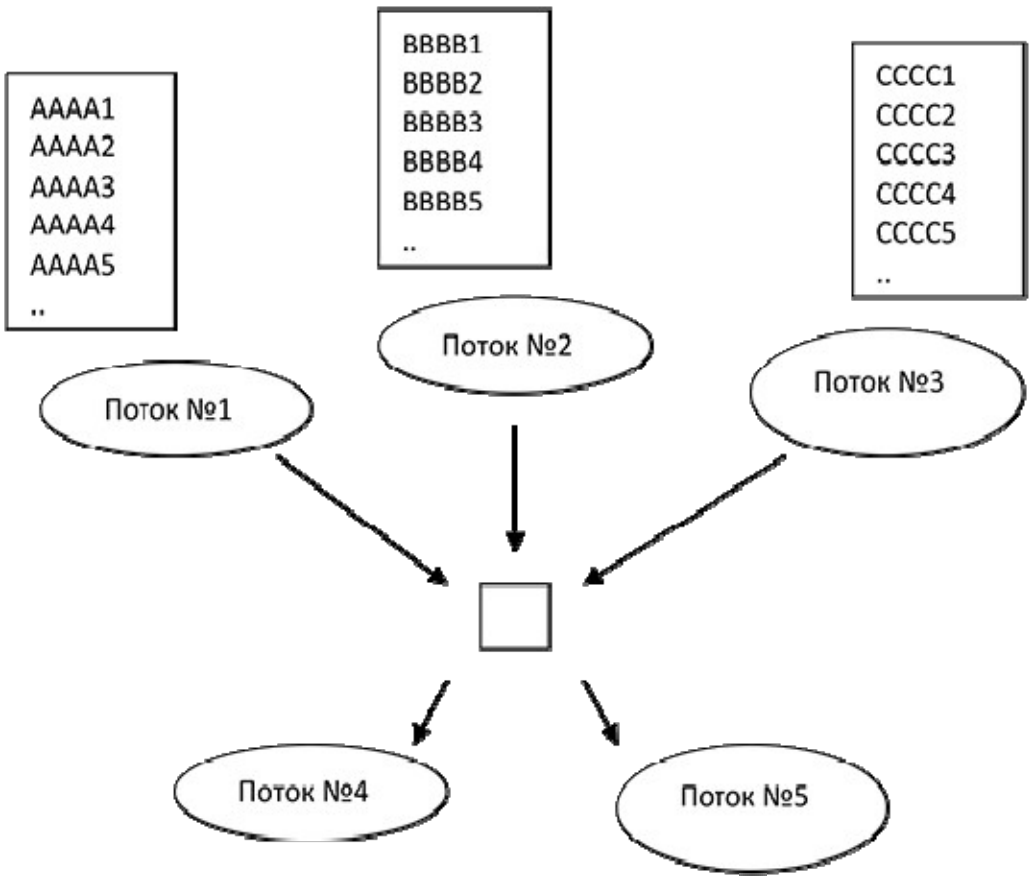


Рисунок 9 – Схема работы приложения

1. Реализуйте взаимодействие потоков-читателей и потоков-писателей с общим буфером без каких-либо средств синхронизации. Проиллюстрируйте проблему совместного доступа. Почему возникает проблема доступа?

2. Реализуйте доступ "читателей" и "писателей" к буферу с применением следующих средств синхронизации:

   a. блокировки (*lock*);

   b. сигнальные сообщения (*ManualResetEvent*, *AutoResetEvent*, *ManualResetEventSlim*);

   c. семафоры (*Semaphore*, *SemaphoreSlim*);

   d. атомарные операторы (*Interlocked*).

3. Исследуйте производительность средств синхронизации при разном числе сообщений, разном объеме сообщений, разном числе потоков.

Сделайте выводы об эффективности применения средств синхронизации.

Требования к отчету Отчет должен содержать:

1. информацию о студенте – ФИО, группа;

2. цель работы;

3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);

4. выводы по итогам выполнения п. 1 задания;

5. результаты в табличной форме по итогам выполнения п. 3 задания (форма таблицы – на ваше усмотрение);

6. выводы по п.4 задания;

7. исходный код, полученный при выполнении пп. 1 и 2 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. Что такое синхронизация?
3. Что такое буфер?
4. Что такое одноэлементный буфер?
5. К чему может привести отсутствие средств синхронизации при обращении к буферу?

ЛАБОРАТОРНАЯ РАБОТА №5 СИНХРОНИЗАЦИЯ ПРИОРИТЕТНОГО ДОСТУПА К МНОГОЭЛЕМЕНТНОМУ БУФЕРУ

Цель работы

Реализовать работу нескольких потоков, имеющих различный приоритет, с многоэлементным буфером.

Основные теоретические положения

В качестве многоэлементного буфера используется кольцевой массив. Он представляет собой обычный массив размера *n*. Буфер называется кольцевым, так как при смещении текущего индекса после крайнего элемента следует первый. Для доступа к буферу используются два индекса: один для чтения и один для записи. Такая организация обеспечивает независимость операций чтения и записи – если в массиве есть свободные элементы и есть заполненные элементы, то операции чтения и записи могут производиться одновременно без каких-либо средств синхронизации.

В начале работы буфер является пустым – оба индекса указывают на первый элемент. При осуществлении операций чтения или записи соответствующие индексы смещаются (рис. 10).

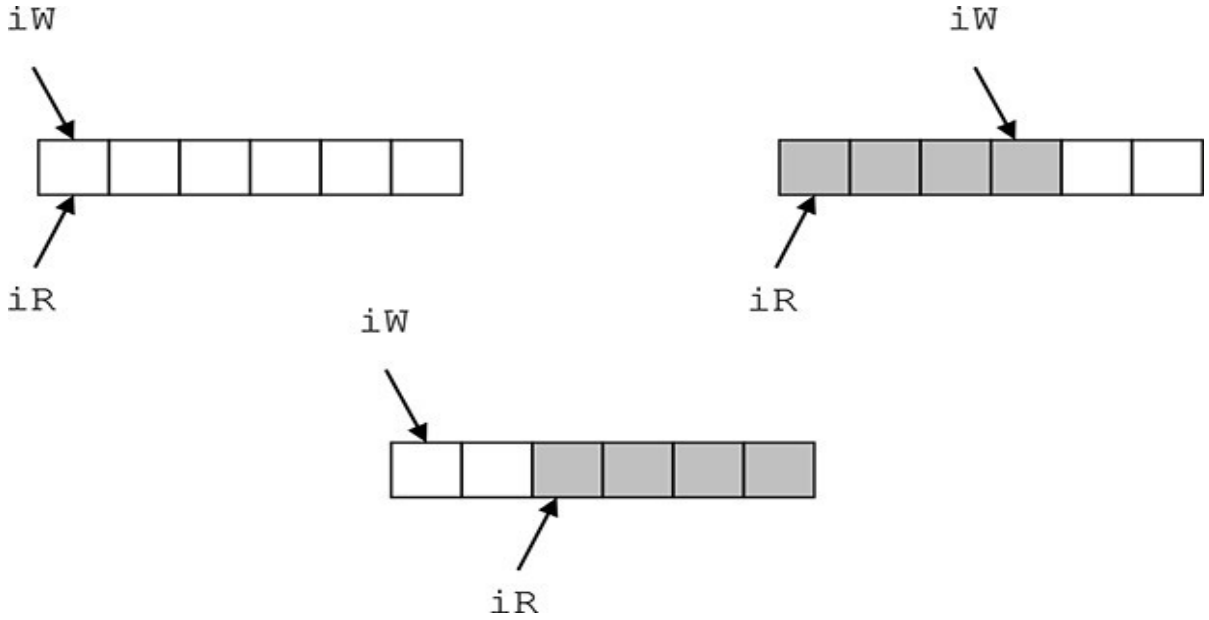


Рисунок 10 – Многоэлементный буфер

Операция чтения блокируется, если буфер пуст, запись при этом разрешена. Операция записи блокируется, если буфер полностью заполнен, чтение при этом разрешено. Равенство индексов чтения и записи является признаком и занятости буфера, и пустоты. Чтобы различать эти ситуации необходимо контролировать, какая операция привела к равенству индексов.

Если операция записи, то буфер заполнен (рис. 11).

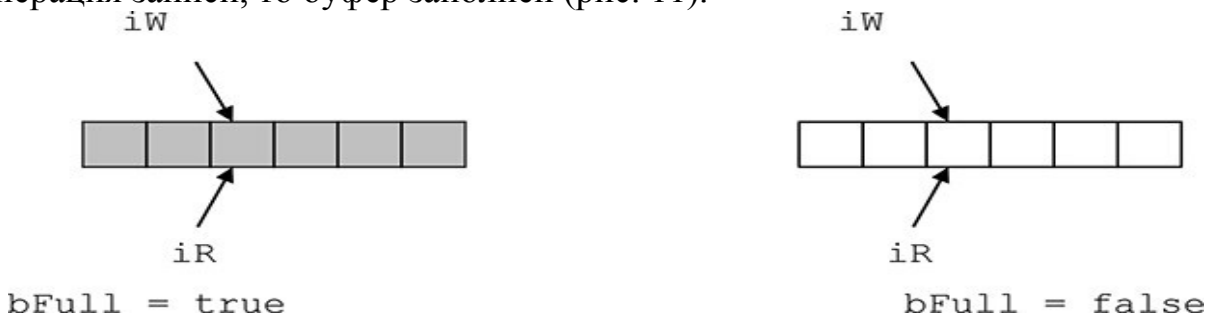


Рисунок 11 – Блокирование буфера

Операции чтения и записи могут быть реализованы следующим образом:

```
bool Write(string Msg)
{       if(bFull)
return false;
buffer[iW] = Msg;       iW
= (iW + 1) % n;
    // Если индексы совпали после записи,
    // буфер заполнен
if(iW == iR)           bFull
= true;       return true;
}
bool Read(ref string Msg)
{
    //Если индексы совпадают, но не после опер.
записи
    // буфер пуст       if(iW
== iR && !bFull)
return false;       Msg =
buffer[iR];       iR = (iR + 1)
% n;
    // Если буфер был заполнен, то снимаем отметку
if(bFull)            bFull = false;       return true;
}
```

Главный поток контролирует статус завершения операций чтения и записи. Если операция чтения не выполнена, то поток читателя блокируется.

Ситуация усложняется, если доступ к буферу осуществляют несколько читателей и несколько писателей. Один из вариантов решения проблемы – добавить конструкции критической секции в функции чтения и записи.

Другой подход заключается в реализации схемы «*управляющий–рабочие*», где управляющий контролирует все операции, требующие синхронизации.

Рабочие потоки (читатели и писатели) обращаются к управляющему (основной поток) с сигналом о готовности осуществлять операцию чтения или записи. Управляющий поток фиксирует обращения читателей и писателей, вычисляет текущие индексы для чтения и записи, контролирует состояние буфера (полностью заполнен или полностью пуст), выбирает читателя и писателя, которым разрешает доступ. Операции чтения и записи по корректным индексам, полученным от управляющего потока, осуществляются читателями и писателями уже без контроля.

Взаимодействие рабочих и управляющего удобно организовать с помощью сигнальных сообщений типа *ManualResetEventSlim*.

Сигналы о готовности *evReadyToRead*, *evReadyToWrite* генерируют читатели и писатели, готовые осуществлять операции с буфером. Управляющий контролирует состояние сигналов у каждого рабочего.

Сигналы о возможности операций чтения и записи *evStartReading*, *evStartWriting* генерируются управляющим потоком конкретным читателям и писателям. Перед генерацией сигналов управляющий вычисляет индекс чтения или записи и сохраняет его в индивидуальной ячейке конкретного рабочего.

Такая организация взаимодействия позволяет достаточно легко изменять правила доступа: вводить приоритеты читателей и писателей, учитывать время обращения к управляющему потоку и обеспечивать "справедливость" доступа в плане очередности.

```
void ReaderThread(int iReader,
ManualResetEventSlim evReadyToRead,
  ManualResetEventSlim evStartReading)
{
    // Инициализация внутреннего буфера      var
Messages = new List<string>();
    // Рабочий цикл чтения      while(true)
    {
        // Сигнализирует о готовности
evReadyToRead.Set();          // Ждем
сигнала от менеджера
evStartReading.Wait();
        // Разрешено чтение по текущему индексу
int k = ReadIndexCopy[iReader];
Messages.Add(buffer[k]);          // Сбрасываем
сигнал о чтении          evStartReading.Reset();
        // Проверяем статус завершения работы
if (finish) break;
    }
}
```

```
// Код писателя практически идентичен коду читателя
void WriterThread(int iWriter,
ManualResetEventSlim evReadyToWrite,
  ManualResetEventSlim evStartWriting)
{
    // Инициализация массива сообщений писателя
    Messages = ..      //
Рабочий цикл записи
while(true)
    {
        // Сигнализируем о готовности менеджеру
evReadyToWrite.Set();          // Ждем сигнала от
менеджера          evStartWriting.Wait();
        // Разрешена запись по текущему индексу
k = WriteIndexCopy[iWriter];          buffer[k] =
Messages[j];
        // Проверяем статус завершения работы
if (finish || j >= Messages.Length)
break;          j++
    }
}

// Код менеджера
void Manager(int nReaders, int nWriters)
{
    // Запуск читателей      for(int
i=0; i<nReaders; i++)
    {
        evReadyToRead[i] =
          new ManualResetEventSlim(false);
        evStartReading[i] =             new
ManualResetEventSlim(false);
tReaders[i] = new Task( () =>
Reader(i, evReadyToRead[i],
evStartReading[i]));
tReaders[i].Start();
    }
    // Запуск писателей
    for(int i=0; i < nWriters; i++)
    {
```

```
        var evReadyToWrite[i] =            new
ManualResetEventSlim(false);          var
evStartWriting[i] =             new
ManualResetEventSlim(false);
tWriters[i] = new Task( () =>
Writer(i, evReadyToWrite[i],
evStartWriting[i]));
tWriters[i].Start();
    }
    // Рабочий цикл       while(true)
    {
        // Если в буфере есть свободные ячейки
        // пытаемся обработать готовых писателей
if(!bFull)
        {
            // Получаем текущий индекс записи
iW = GetBufferWriteIndex();              if(iW !=
-1)
            {
                // Устанавливаем писателя,
// которому разрешаем работать
iWriter = GetWriter();                   if
(iWriter != -1)
                {
                    // Сбрасываем сигнал готовности
// выбранного писателя
evReadyToWrite[iWriter].Reset();                 //
Сохраняем копию индекса для записи
                    ReadIndexCopy[iWriter] = iW;
// Разрешаем писателю начать работу
evStartWriting[iWriter].Set();
                }
}              else
bFull = true;
        }
        // Если буфер не пуст, пытаемся
// обработать готовых писателей          if(!
bEmpty)
        {
            // Получаем текущий индекс для чтения
iR = GetBufferReadIndex();              if(iR != -1)
```

```
            {
                //Устанавливаем готового читателя
iReader = GetWriter();                  if (iReader !
= -1)
                {
                    evReadyToRead[iReader].Reset();
WriteIndexCopy[iReader] = iR;
evStartReading[iReader].Set();
                }
}              else
                bEmpty = false;
        }
    }
}
// Код функции получения номера готового писателя
// с учетом приоритетов  int
GetWriter()
{
    // Устанавливаем готовых писателей
var ready = new List<int>();      for(int
i=0; i<nWriter; i++)
if(evReadyToWrite[i].IsSet())
ready.Add(i);      if(ready.Count == 0)
return -1;      return ready.OrderBy(i =>
        WriterPriority[i]).First();
}
```

Задание

Несколько потоков работают с общим многоэлементным буфером. Потоки делятся на «*читателей*» и «*писателей*», каждый поток обладает приоритетом. Писатели осуществляют запись в буфер, если есть свободные ячейки. Читатели извлекают содержимое буфера, если есть заполненные ячейки. Работа приложения заканчивается после того, как все сообщения писателей будут обработаны читателями через общий буфер. В качестве буфера используется «*кольцевой массив*».

1. Реализуйте синхронизированное взаимодействие читателей и писателей с учетом приоритета. Аргументируйте выбор средств синхронизации (рис. 12).
2. Обеспечьте, чтобы вывод программы включал: время работы каждого писателя и читателя; число сообщений, обработанных каждым писателем и читателем.

3. Выполните программу при разных параметрах: разном числе писателей и читателей, разном объеме сообщений, разных приоритетах потоков. Результаты выполнений представьте в табличной форме (форма таблицы – на ваше усмотрение).

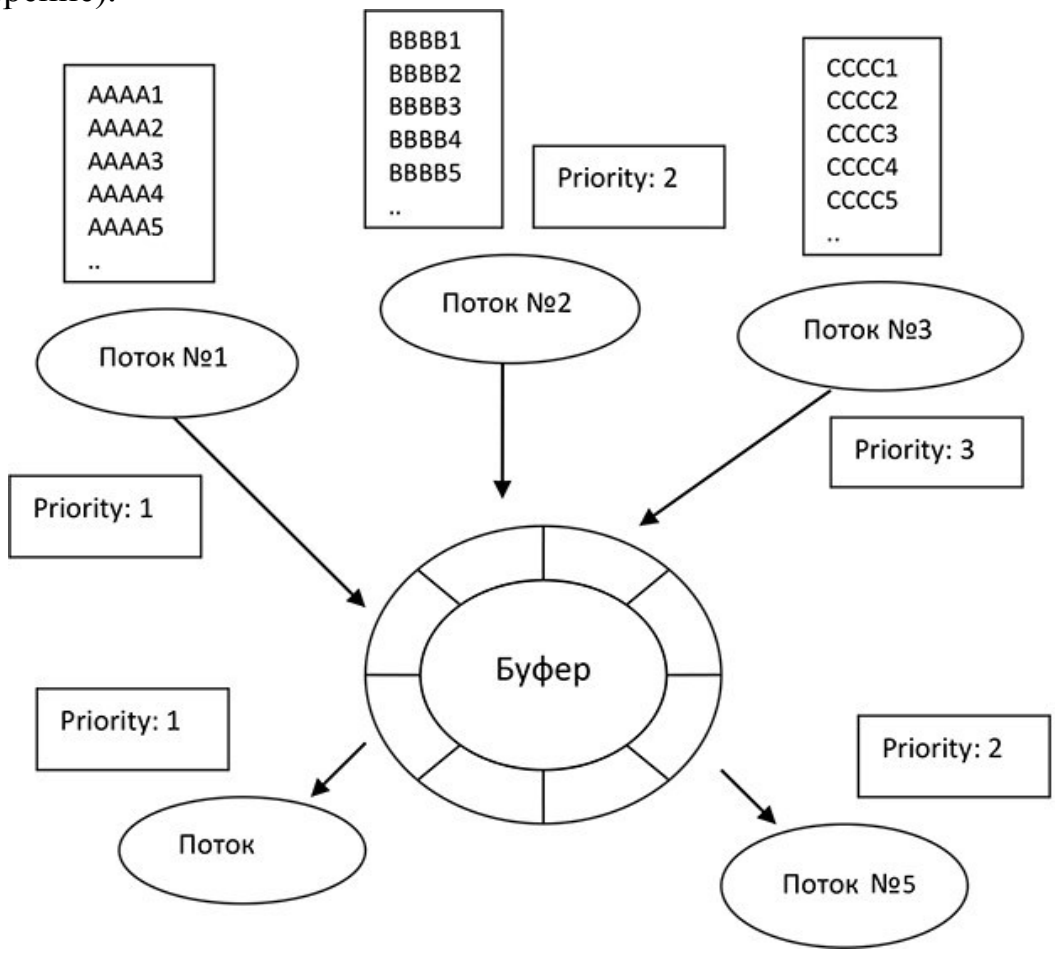


Рисунок 12 – Схема взаимодействия

Порядок выполнения работы

1. Изучить основные теоретические положения, и выполнить предоставленные задания;
2. По результатам задания 3 заполнить таблицу (форма таблицы – на ваше усмотрение).

Требования к отчету

Отчет должен содержать:

1. информацию о студенте – ФИО, группа;
2. цель работы;
3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);
4. результаты в табличной форме по итогам выполнения п. 3 задания

(форма таблицы – на ваше усмотрение);

5. исходный код, полученный при выполнении пп. 1 и 2 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. Что из себя представляет кольцевой массив?

ЛАБОРАТОРНАЯ РАБОТА №6. КЛЕТОЧНАЯ МОДЕЛЬ «ИГРА ЖИЗНЬ»

Цель работы

Создать приложение, реализующее клеточную модель «Игра Жизнь» Дж. Конвея.

Основные теоретические положения

Клеточная модель представляет собой множество клеток (ячеек таблицы), принимающих одно из нескольких состояний. Состояние каждой клетки определяется состоянием её ближайших соседей. Одной из известных моделей является «Игра Жизнь» математика Дж. Конвея.

В модели Конвея каждая клетка может находиться в двух состояниях: живая или неживая. Состояние клетки на следующем шаге определяется потенциалом клетки (числом живых соседних клеток):

- если потенциал клетки равен двум, то клетка сохраняет свое состояние;
- если потенциал равен трем, то клетка оживает;
- если потенциал меньше двух или больше трех, то клетка погибает.

Правила изменения состояния клетки можно описать следующим лямбда-выражением:

```
var lifeRules = new Func<int, bool, bool>((p, state)
=>  {
    if(p == 3)   return true;
else         if (p == 2) return
state;          else return false;
});
```

Последовательный алгоритм расчета представляет собой расчет состояния каждой клетки:

```
LifeTable tableNew = new LifeTable;
for(int i=0; i < Height; i++)      for(int
j=0; j < Width; j++)
    {
        p = CalcPotential(table[i,j]);
        tableNew[i, j] = lifeRules(p, table[i, j]);
    }
```

```
table = tableNew;
```

Потенциал клетки вычисляется по восьми её ближайшим соседям.

```
int CalcPotential(int i, int j)
{       int p=0;
    for(int x = i-1; x <= i + 1; x++)
for(int y = j-1; y <= j + 1;y++)
        {
                if (x < 0 || y < 0 || x >= Height ||
y   >=   Width   ||   (x   ==   i   &&   y   ==   j))
continue;               if(table[x,y])  p++;
        }       return
p;
}
```

Вычисления новых состояний для каждой клетки являются независимыми и могут выполняться параллельно.

```
Parallel.For(0, Height, (i) => {
for(int j=0; j < Width; j++)
    {
        p = CalcPotential(table[i,j]);
        tableNew[i, j] = lifeRules(p, table[i, j]);
    }
});
```

Также можно изменить порядок расчета и распараллелить внешний цикл по столбцам:

```
Parallel.For(0, Width, (j) => {       for(int
i=0; i < Height; i++)
    {
        p = CalcPotential(table[i,j]);
        tableNew[i, j] = lifeRules(p, table[i, j]);
    }
}
```

Вывод таблицы состояний клеточной модели может быть следующим.

Порядок выполнения работы

Изучить основные теоретические положения, и выполнить предоставленные задания.

Задание

1. Реализуйте Windows-приложение, которое последовательно отображает состояния клеточной модели «Игра Жизнь».
2. Реализуйте последовательный алгоритм расчета состояний модели (рис. 13).
3. Реализуйте параллельные алгоритмы расчета состояний модели. Для распараллеливания используйте задачи (*tasks*) или метод *Parallel.For*.
4. Реализуйте возможность отмены расчета с помощью объекта *CancellationToken*.
5. Выполните анализ эффективности разработанных алгоритмов.

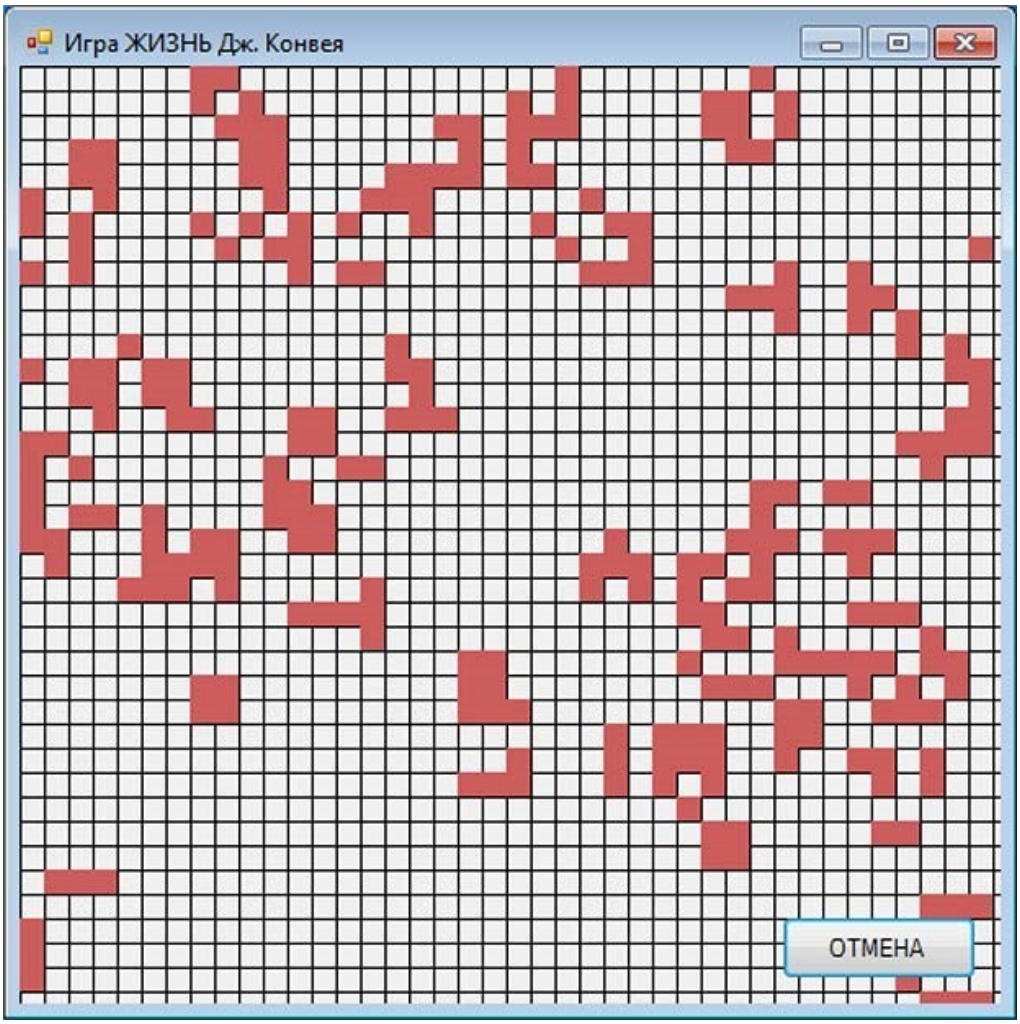


Рисунок 13 – Примерный интерфейс игры

Требования к отчету

Отчет должен содержать:

1. информацию о студенте – ФИО, группа;
2. цель работы;

3. информацию о компьютере, на котором выполнялись задания (наименование процессора, количество ядер, тактовая частота);
4. результаты анализа по итогам выполнения п. 5 задания; 5. исходный код, полученный при выполнении пп. 2-4 задания.

Контрольные вопросы

1. Прокомментируйте исходный код разработанной программы.
2. В чем заключается клеточная модель «Игра Жизнь» Дж. Конвея?

ЛАБОРАТОРНЫЕ РАБОТЫ №7. ПРОЦЕССЫ В ОПЕРАЦИОННОЙ СИСТЕМЕ UNIX.

Цели занятия

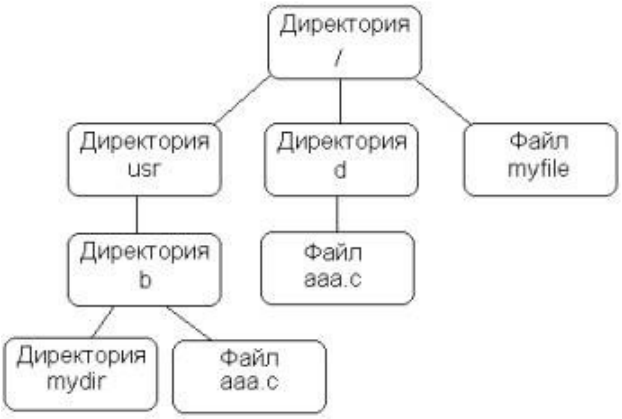
- Дать понятие об иерархии процессов в UNIX.
- Дать понятие о содержании контекста процесса в UNIX.
- Научить создавать новый процесс.
- Научить запускать новую программу.
- Дать понятие об использовании переменных среды и аргументов командной строки.
- Студент должен осознать разницу между системными вызовами fork() и exec().
- Студент должен понять, как завершается программа, написанная на C, даже если она не вызывает явно функцию exit()

Практические работы

- Написание, компиляция и запуск программы с системными вызовами getpid() и getppid().
- Прогон программы с использованием вызова fork(), где порожденный процесс делает то же самое, что и родитель.
- Написание, компиляция и запуск программы с использованием вызова fork() с разным поведением процессов ребенка и родителя.
- Написание, компиляция и запуск программы с распечаткой значений переменных среды и аргументов командной строки.
- Прогон программы с использованием системного вызова exec().
- Написание, компиляция и запуск программы для изменения пользовательского контекста в порожденном процессе.

1.Понятие процесса в UNIX. Его контекст.

Все построение операционной системы UNIX основано на использовании концепции процессов. Контекст процесса складывается из пользовательского контекста и контекста ядра, как изображено на рисунке:



Под пользовательским контекстом процесса понимают код и данные, расположенные в адресном пространстве процесса. Все данные подразделяются на инициализируемые неизменяемые данные (например, константы), инициализируемые изменяемые данные (все переменные, начальные значения которых присваиваются на этапе компиляции), неинициализируемые изменяемые данные (все статические переменные, которым не присвоены начальные значения на этапе компиляции), стек пользователя и данные, расположенные в динамически выделяемой памяти (например, с помощью стандартных библиотечных С функций malloc, calloc и realloc). Исполняемый код и инициализируемые данные составляют содержимое файла программы, который исполняется в контексте процесса. Пользовательский стек используется при работе процесса в пользовательском режиме (user-mode).

Под понятием контекст ядра объединяются системный контекст и регистровый контекст. Мы будем выделять в контексте ядра стек ядра, который используется при работе процесса в режиме ядра (kernel mode), и данные ядра, хранящиеся в структурах, являющихся аналогом блока управления процессом - PCB. На этом занятии достаточно знать, что в данные ядра входят: идентификатор пользователя - UID, групповой идентификатор пользователя - GID, идентификатор процесса - PID, идентификатор родительского процесса - PPID.

2.Идентификация процесса.

Каждый процесс в операционной системе получает свой собственный уникальный идентификационный номер PID (Process IDentificator). При создании нового процесса операционная система пытается присвоить ему свободный номер больший, чем у процесса, созданного перед ним. Если таких свободных номеров не оказывается (например, мы достигли максимально возможного номера для процесса), то операционная система выбирает минимальный из всех свободных номеров. В операционной системе Linux присвоение идентификационных номеров процессов начинается с номера 0, который получает процесс *kernel* при старте операционной системы. Максимально возможное значение для номера процесса в Linux на базе процессоров Intel составляет 231-1.

3.Состояния процесса. Краткая диаграмма состояний.

Краткая диаграмма состояний процессов в операционной системе UNIX изображена на рисунке:



Как видим, состояние процесса *исполнение* расщепилось на 2 состояния: *исполнение в режиме ядра* и *исполнение в режиме пользователя*. В состоянии *исполнение в режиме пользователя* процесс выполняет прикладные инструкции пользователя. В состоянии *исполнение в режиме ядра* выполняются инструкции ядра операционной системы в контексте текущего процесса (например, при обработке системного вызова или прерывания). Из состояния *исполнение в режиме пользователя* процесс не может непосредственно перейти в состояния *ожидание*, готовность и закончил *исполнение.* Такие переходы возможны только через промежуточное состояние *исполняется в режиме ядра*. Точно также запрещен прямой переход из состояния *готовность* в состояние *исполнение в режиме пользователя.*

Приведенная выше диаграмма состояний процессов в Linux не является полной. Она показывает только состояния, для понимания которых достаточно уже полученных знаний. Полную диаграмму состояний процессов в операционной системе UNIX можно найти в книге Баха "Архитектура операционной системы UNIX" (рисунок 6.1.).

4.Иерархия процессов.

В операционной системе UNIX все процессы кроме одного, создающегося при старте операционной системы, могут быть порождены только какими-либо другими процессами. В качестве процесса прародителя всех остальных процессов в разных UNIX-ообразных системах могут выступать процессы с номерами 1 или 0. В операционной системе Linux таким родоначальником, существующим только при загрузке, является процесс *kernel* с идентификатором 0.

Таким образом, все процессы в UNIX связаны отношениями процесс-родитель - процесс-ребенок, образуя генеалогическое дерево процессов. Для сохранения целостности генеалогического дерева в ситуациях, когда процесс-родитель завершает свою работу до завершения выполнения процесса-ребенка, идентификатор родительского процесса в данных ядра процесса-ребенка (PPID - Parent Process IDentificator) изменяет свое значение на значение 1, соответствующее идентификатору процесса init, время жизни которого определяет время функционирования операционной системы. Тем самым процесс init как бы усыновляет осиротевшие процессы. Наверное, логичнее было бы изменять PPID не на значение 1, а на значение идентификатора ближайшего существующего процесса-прародителя умершего процесса-родителя, но в UNIX почему-то такая схема реализована не была.

5.Системные вызовы getppid() и getpid().

Данные ядра, находящиеся в контексте ядра процесса, не могут быть прочитаны процессом непосредственно. Для получения информации о них процесс должен совершить соответствующий системный вызов. Значение идентификатора текущего процесса может быть получено с помощью системного вызова getpid(), а значение идентификатора родительского процесса для текущего процесса - с помощью системного вызова getppid().

6.Создание процесса в UNIX. Системный вызов fork().

В операционной системе UNIX новый процесс может быть порожден единственным способом - с помощью системного вызова fork(). При этом вновь созданный процесс будет являться практически полной копией родительского процесса. У порожденного процесса по сравнению с родительским процессом (на уровне уже полученных знаний) изменяются значения следующих параметров:

- идентификатор процесса - PID;
- идентификатор родительского процесса - PPID;

Дополнительно к ним может измениться поведение порожденного процесса по отношению к некоторым сигналам.

В процессе выполнения системного вызова fork() порождается копия родительского процесса и возвращение из системного вызова будет происходить уже как в родительском, так и в порожденном процессах. Этот системный вызов является единственным, который вызывается один раз, а при успешной работе возвращается два раза (один раз в процессе-родителе и один раз в процессе-ребенке)! После выхода из системного вызова оба процесса продолжают выполнение регулярного пользовательского кода, следующего за системным вызовом.

7.Прогон программы с fork() с одинаковой работой родителя и ребенка.

Для иллюстрации вышесказанного давайте рассмотрим программу, находящуюся в файле \\ИБ3\БОС\linux\03-1.c, откомпилируем ее и запустим на исполнение.

8.Системный вызов fork() (продолжение).

Для того, чтобы после возвращения из системного вызова fork() процессы могли определить, кто из них является ребенком, а кто родителем, и, соответственно, по-разному организовать свое поведение, он возвращает в них разные значения. При успешном создании нового процесса в процесс-родитель возвращается положительное значение равное идентификатору процесса-ребенка. В процесс-ребенок же возвращается значение 0. Если по какой-либо причине создать новый процесс не удалось, то системный вызов вернет в инициировавший его процесс значение -1. Таким образом, общая схема организации различной работы процесса-ребенка и процесса-родителя выглядит так:

```
pid = fork();
if(pid == -1){
...
/* ошибка */
...
} else if (pid == 0){
...
/* ребенок */
...
} else {
...
/* родитель */
...
```

}

9.Завершение процесса. Функция exit().

Существует два способа корректного завершения процесса в программах, написанных на языке С. Первый способ мы использовали до сих пор: процесс корректно завершался по достижении конца функции **main()** или при выполнении оператора **return** в функции **main()**, второй способ применяется при необходимости завершить процесс в каком-либо другом месте программы. Для этого применяется функция **exit()** из стандартной библиотеки функций для языка С. При выполнении этой функции происходит сброс всех частично заполненных буферов ввода-вывода с закрытием соответствующих потоков, после чего инициируется системный вызов прекращения работы процесса и перевода его в состояние *закончил исполнение.*

Возврата из функции в текущий процесс не происходит и функция ничего не возвращает.

Значение параметра функции exit() - кода завершения процесса - передается ядру операционной системы и может быть затем получено процессом, породившим завершившийся процесс. На самом деле при достижении конца функции **main()** также неявно вызывается эта функция со значением параметра 0.

Если процесс завершает свою работу раньше, чем его родитель, и родитель явно не указал, что он не хочет получать информацию о статусе завершения порожденного процесса, то завершившийся процесс не исчезает из системы окончательно, а остается в состоянии закончил исполнение либо до завершения процесса-родителя, либо до того момента, когда родитель соблаговолит получить эту информацию. Процессы, находящиеся в состоянии *закончил исполнение*, в операционной системе принято называть процессами-зомби (zombie, defunct).

10.Параметры функции main() в языке С. Переменные среды и аргументы командной строки.

У функции **main()** в языке программирования С существует три параметра, которые могут быть переданы ей операционной системой. Полный прототип функции **main()** выглядит следующим образом:

int main(int argc, char *argv[], char *envp[]);

Первые два параметра при запуске программы на исполнение командной строкой позволяют узнать полное содержание командной строки. Вся командная строка рассматривается как набор слов, разделенных пробелами. Через параметр **argc** передается количество слов в командной строке, которой была запущена программа. Параметр **argv** является массивом указателей на отдельные слова. Так, например, если программа была запущена командой

a.out 12 abcd

то значение параметра argc будет равно 3, **argv[0]** будет указывать на имя программы - первое слово - **"a.out", argv[1]** - на слово "12", **argv[2]** - на слово **"abcd**". Заметим, что, так как имя программы всегда присутствует на первом месте в командной строке, то **argc** всегда больше 0, а **argv[0**] всегда указывает на имя запущенной программы.

Анализируя в программе содержимое командной строки, мы можем предусмотреть ее различное поведение в зависимости от слов следующих за именем программы. Таким образом, не внося изменений в текст программы, мы можем заставить ее работать по-разному от запуска к запуску. Например, компилятор **gcc**, вызванный командой **gcc 1**.c будет генерировать исполняемый файл с именем **a.out**, а при вызове командой **gcc 1.c -o 1**.**exe** - файл с именем **1.exe.**

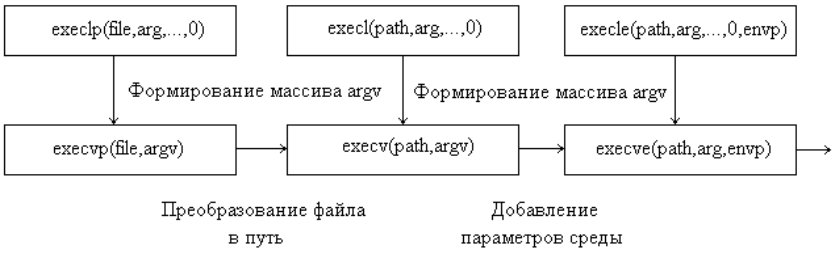
Третий параметр - **envp** - является массивом указателей на параметры окружающей среды процесса. Начальные параметры окружающей среды процесса задаются в специальных конфигурационных файлах для каждого пользователя и устанавливаются при входе пользователя в систему. В последующем они могут быть изменены с помощью специальных команд операционной системы UNIX. Каждый параметр имеет вид: ***переменная=строка***. Такие

переменные используются для изменения долгосрочного поведения процессов, в отличие от аргументов командной строки. Например, задание параметра **TERM=vt100** может говорить процессам, осуществляющим вывод на экран дисплея, что работать им придется с терминалом **vt100**. Меняя значение переменной среды **TERM**, например на **TERM=console**, мы говорим таким процессам, что они должны изменить свое поведение на вывод для системной консоли.

Размер массива аргументов командной строки в функции **main()** мы получали в качестве ее параметра. Так как для массива ссылок на параметры окружающей среды такого параметра нет, то его размер определяется другим способом. Последний элемент этого массива содержит указатель **NULL**.

11.Изменение пользовательского контекста процесса. Семейство функций для системного вызова exec().

Для изменения пользовательского контекста процесса используется системный вызов exec(), который пользователь не может вызвать непосредственно. Вызов exec() заменяет пользовательский контекст текущего процесса на содержимое некоторого исполняемого файла и устанавливает начальные значения регистров процессора (в том числе устанавливает программный счетчик на начало загружаемой программы). Этот вызов требует для своей работы задания имени исполняемого файла, аргументов командной строки и параметров окружающей среды. Для осуществления вызова программист может воспользоваться одной из 6 функций: execlp(), execvp(), execl(), execv(), execle(), execve(), отличающихся друг от друга представлением параметров, необходимых для работы системного вызова exec(). Взаимосвязь указанных выше функций изображена на рисунке.



Поскольку системный контекст процесса при вызове exec() остается практически неизменным, то большинство атрибутов процесса, доступных пользователю через системные вызовы (PID, UID, GID, PPID и другие, смысл которых будет становиться понятным по мере углубления наших знаний на дальнейших занятиях), также не изменяется после запуска новой программы

Важно понимать разницу между системными вызовами fork() и exec() Системный вызов fork() создает новый процесс, у которого пользовательский контекст совпадает с пользовательским контекстом процесса-родителя. Системный вызов exec() изменяет пользовательский контекст текущего процесса, не создавая новый процесс.

12.Прогон программы с использованием системного вызова exec()

Для иллюстрации использования системного вызова exec() давайте рассмотрим программу в файле \\ИБ3\БОС\linux\03-2.c, откомпилируем ее и запустим на исполнение. Поскольку в результате должно распечататься содержимое файла с именем 03-2.c, такой файл при запуске должен присутствовать в текущей директории.

Задание:

1. В качестве примера на использование системных вызовов getpid() и getppid() вам предлагается самостоятельно написать программу, печатающую значения PID и PPID для текущего процесса.

2. Изменить программу с fork(), приведенную выше так, чтобы родитель и ребенок совершали разные действия (какие не важно).
3. В качестве примера студентам предлагается самостоятельно написать программу, распечатывающую значения аргументов командной строки и параметров окружающей среды для текущего процесса.
4. Для закрепления полученных знаний модифицируйте программу из пункта 2 так, чтобы порожденный процесс запускал на исполнение новую программу.

Контрольные вопросы

1. Понятие процесса в UNIX, его контекст.
2. Идентификация процесса.
3. Состояния процесса. Краткая диаграмма состояний.
4. Иерархия процессов.
5. Системные вызовы getpid(), getppid().
6. Создание процесса в UNIX. Системный вызов fork().
7. Завершение процесса. Функция exit().
8. Параметры функции main() в языке С. Переменные среды и аргументы командной строки.
9. Изменение пользовательского контекста процесса. Семейство функций для системного вызова exec().

ЛАБОРАТОРНАЯ РАБОТА №8. ОРГАНИЗАЦИЯ ВЗАИМОДЕЙСТВИЯ ПРОЦЕССОВ ЧЕРЕЗ PIPE И FIFO В UNIX.

Цели занятия

- Дать понятие о потоке данных в UNIX.
- Научить использовать системные вызовы open(), read(), write(), close() при работе с файлами.
- Научить создавать pipe и использовать его для связи между родственными процессами.
- Научить создавать FIFO и использовать его для связи между неродственными процессами.
- Студент должен четко осознать, что содержимое FIFO хранится в оперативной памяти, а не на диске.

Практические работы

- Прогон программы для записи в файл, используя вызовы open(), write(), close().
- Написание, компиляция и запуск программы для чтения из файла, используя вызовы open(), read(), close()
- Прогон программы для pipe в одном процессе.
- Прогон программы для организации однонаправленной связи между родственными процессами через pipe.
- Написание, компиляция и запуск программы для организации двунаправленной связи между родственными процессами через pipe.
- Прогон программы с FIFO в родственных процессах.
- Написание, компиляция и запуск программы с FIFO в неродственных процессах.
- Неработающий пример для связи процессов на различных компьютерах.

1.Понятие о потоке ввода-вывода.

Среди всех категорий средств коммуникации наиболее употребительными являются каналы связи, обеспечивающие достаточно безопасное и достаточно информативное общение процессов.

Существует две модели передачи данных по каналам связи - поток ввода-вывода и сообщения. Из них более простой идеологически является потоковая модель, в которой операции передачи/приема информации вообще не интересуются содержимым того, что передается или принимается. Вся информация в канале связи рассматривается как непрерывный поток байт, не обладающий никакой внутренней структурой.

2.Понятие о работе с файлами через системные вызовы и стандартную библиотеку ввода-вывода для языка С.

Потоковая передача информации может осуществляться не только между процессами, но и между процессом и устройством ввода-вывода, например между процессом и диском, на котором данные представляются в виде файла. Поскольку понятие файла должно быть знакомо изучающим этот курс, а системные вызовы, использующиеся для потоковой работы с файлом, во многом соответствуют системным вызовам, применяемым для потокового общения процессов, мы начнем наше рассмотрение именно с механизма потокового обмена между процессом и файлом.

Из курса программирования на языке С вам должны быть известны, как мы надеемся, функции работы с файлами из стандартной библиотеки ввода-вывода такие, как fopen(), fread(), fwrite(), fprintf(), fscanf(), fgets() и т.д. Эти функции входят как неотъемлемая часть в стандарт ANSI на язык С и позволяют программисту получать информацию из файла или записывать ее в файл при условии, что программист обладает определенными знаниями о содержимом передаваемых данных. Так, например, функция fgets() используется для ввода из файла последовательности символов, заканчивающейся символом '\n' - перевод каретки; функция fscanf() производит ввод информации, соответствующей заданному формату, и т. д. С точки

зрения потоковой модели операции, определяемые функциями стандартной библиотеки ввода-вывода, не являются потоковыми операциями, так как каждая из них требует наличия некоторой структуры передаваемых данных.

В операционной системе UNIX эти функции представляют собой надстройку - сервисный интерфейс - над системными вызовами, осуществляющими прямые потоковые операции обмена информацией между процессом и файлом и не требуюшими никаких априорных знаний о том, что она содержит. Чуть позже мы кратко познакомимся с системными вызовами *open(), read(), write() и close(),* которые применяются для такого обмена, но сначала нам нужно ввести еще одно понятие - понятие *файлового дескриптора*.

3.Файловый дескриптор.

На лекции мы говорили, что информация о файлах, используемых процессом, входит в состав его системного контекста и хранится в его блоке управления - PCB. В операционной системе UNIX можно упрощенно полагать, что информация о файлах, с которыми процесс осуществляет операции потокового обмена, наряду с информацией о потоковых линиях связи, соединяющих процесс с другими процессами и устройствами ввода-вывода, хранится в некотором массиве, получившем название таблицы открытых файлов или таблицы файловых дескрипторов. Индекс элемента этого массива, соответствующий определенному потоку ввода-вывода, получил название файлового дескриптора для этого потока. Таким образом, файловый дескриптор представляет собой небольшое целое неотрицательное число, которое для текущего процесса в текущий момент времени однозначно определяет некоторый действующий канал ввода-вывода. Некоторые файловые дескрипторы на этапе старта любой программы ассоциируются со стандартными потоками ввода-вывода. Так, например, файловый дескриптор 0 соответствует стандартному потоку ввода, файловый дескриптор 1 - стандартному потоку вывода, файловый дескриптор 2 - стандартному потоку для вывода ошибок. В нормальном интерактивном режиме работы стандартный поток ввода связывает процесс с клавиатурой, а стандартные потоки вывода и вывода ошибок - с текущим терминалом.

Более детально строение структур данных, содержащих информацию о потоках ввода-вывода, ассоциированных с процессом, мы будем рассматривать позже, при изучении организации файловых систем в UNIX.

4.Открытие файла. Системный вызов open().

Файловый дескриптор используется в качестве параметра, описывающего поток ввода-вывода, для системных вызовов, выполняющих операции над этим потоком. Поэтому, прежде чем совершать операции чтения данных из файла и записи их в файл, мы должны поместить информацию о файле в таблицу открытых файлов и определить соответствующий файловый дескриптор. Для этого применяется процедура открытия файла, осуществляемая системным вызовом *open()*.

Системный вызов *open()* использует набор флагов для того, чтобы специфицировать операции, которые предполагается применять к файлу в дальнейшем или которые должны быть выполнены непосредственно в момент открытия файла. Из всего возможного набора флагов на текущем уровне знаний нас будут интересовать только флаги **O_RDONLY, O_WRONLY, O_RDWR, O_CREAT и O_EXCL**. Первые три флага являются взаимоисключающими: хотя бы один из них должен быть употреблен и наличие одного из них не допускает наличия двух других. Эти флаги описывают набор операций, которые, при успешном открытии файла, будут разрешены над файлом в последующем: только чтение, только запись, чтение и запись. Как вам известно, у каждого файла существуют атрибуты прав доступа для различных категорий пользователей. Если файл с заданным именем существует на диске, и права доступа к нему для пользователя, от имени которого работает текущий процесс, не противоречат запрошенному набору операций, то операционная система сканирует таблицу открытых файлов от ее начала к концу в поисках первого свободного элемента, заполняет его и возвращает индекс этого

элемента в качестве файлового дескриптора открытого файла. Если файла на диске нет, не хватает прав или отсутствует свободное место в таблице открытых файлов, то констатируется возникновение ошибки.

В случае, когда мы допускаем, что файл на диске может отсутствовать, и хотим, чтобы он тогда был создан, флаг для набора операций должен использоваться в комбинации с флагом **O_CREAT**. Если файл существует, то все происходит по рассмотренному выше сценарию. Если файла нет - то сначала выполняется создание файла с набором прав, указанном в параметрах системного вызова. Проверка соответствия набора операций объявленным правам доступа может и не производиться (как, например, в Linux).

В случае, когда мы требуем, чтобы файл на диске отсутствовал и был создан в момент открытия, флаг для набора операций должен использоваться в комбинации с флагами **O_CREAT и O_EXCL**.

5.Системные вызовы read(), write(), close().

Для совершения потоковых операций чтения информации из файла и ее записи в файл применяются системные вызовы read() и write(). Мы сейчас не акцентируем внимание на понятии указателя текущей позиции в файле и взаимном влиянии значения этого указателя и поведения системных вызовов. Этот вопрос будет обсуждаться в дальнейшем.

После завершения потоковых операций, процесс должен выполнить операцию закрытия потока ввода-вывода, во время которой произойдет окончательный досброс буферов на линии связи, освободятся выделенные ресурсы операционной системы, и элемент таблицы открытых файлов, соответствующий файловому дескриптору, будет отмечен как свободный. За эти действия отвечает системный вызов close(). Надо отметить, что при завершении работы процесса с помощью явного или неявного вызова функции exit () происходит автоматическое закрытие всех открытых потоков ввода-вывода.

6.Прогон программы для записи информации в файл.

Для иллюстрации вышесказанного давайте рассмотрим программу, находящуюся в файле \\ИБ3\БОС\linux\05-1.c, откомпилируем ее и запустим на исполнение. Обратите внимание на использование системного вызова umask() с параметром 0 для того, чтобы права доступа к созданному файлу точно соответствовали указанным в системном вызове open().

7.Понятие о pipe. Системный вызов pipe().

Наиболее простым способом для передачи информации с помощью потоковой модели между различными процессами или даже внутри одного процесса в операционной системе UNIX является pipe (канал, труба, конвейер). **Важным отличием pip'a от файла является то, что прочитанная информация немедленно удаляется из него и не может быть прочитана повторно**. О pip'е можно думать, как о трубе ограниченной емкости, расположенной внутри адресного пространства операционной системы, доступ к входному и выходному отверстию которой осуществляется с помощью системных вызовов. В действительности pipe представляет собой область памяти, недоступную пользовательским процессам напрямую, зачастую организованную в виде кольцевого буфера (хотя существуют и многочисленные другие виды организации). По буферу при операциях чтения и записи перемещаются два указателя, соответствующие входному и выходному потокам. При этом выходной указатель никогда не может перегнать входной и наоборот. Для создания нового экземпляра такого кольцевого буфера внутри операционной системы используется системный вызов pipe(). При своей работе он организует выделение области памяти под буфер и указатели и заносит информацию, соответствующую входному и выходному потокам данных в два элемента таблицы открытых файлов, связывая тем самым с каждым pipe'ом два файловых дескриптора. Для одного из них разрешена только операция чтения из pip'a, а для другого - только операция записи в pipe. Для выполнения этих операций мы можем использовать те же самые системные вызовы read() и write(), что и при работе с файлами. Естественно, что по окончании необходимости

использования входного или/и выходного потока данных, нам необходимо закрыть соответствующий поток с помощью системного вызова close() для освобождения системных ресурсов. Необходимо отметить, что, когда все процессы, использующие pip'е, закрывают все ассоциированные с ним файловые дескрипторы, операционная система ликвидирует pipe. Таким образом, время существования pip'а в системе не может превышать время жизни процессов, работающих с ним.

8.Прогон программы для pipe в одном процессе.

Достаточно понятной иллюстрацией действий по созданию pip'а, записи в него данных, чтению из него и освобождению выделенных ресурсов может служить программа, организующая работу с pip'ом в рамках одного процесса.

9.Организация связи через pipe между процессом-родителем и процессом-потомком. Наследование файловых дескрипторов при вызовах fork() и exec().

Понятно, что если бы все достоинство pip'ов сводилось к замене функции копирования из памяти в память внутри одного процесса на пересылку информации через операционную систему, то овчинка не стоила бы выделки. Однако таблица открытых файлов наследуется процессом-ребенком при порождении нового процесса системным вызовом fork() и входит в состав неизменяемой части системного контекста процесса при системном вызове exec() (за исключением тех потоков данных, для файловых дескрипторов которых был специальными средствами выставлен признак, побуждающий операционную систему закрыть их при выполнении exec(); однако их рассмотрение выходит за рамки нашего курса). Это обстоятельство позволяет организовать передачу информации через pipe между родственными процессами, имеющими общего прародителя, создавшего pipe.

10.Прогон программы для организации однонаправленной связи между родственными процессами через pipe.

Давайте рассмотрим программу, осуществляющую однонаправленную связь между процессом-родителем и процессом-ребенком, находящуюся в файле \\ИБ3\БОС\linux\05-3.c, откомпилируем ее и запустим на исполнение.

11.Написание, компиляция и запуск программы для организации двунаправленной связи между родственными процессами через pipe..

Pipe принципиально служит для организации однонаправленной или симплексной связи. Если бы в предыдущем примере мы попытались организовать через pipe двустороннюю связь, когда процесс-родитель пишет информацию в pipe, предполагая, что ее получит процесс-ребенок, а затем читает информацию из pip'а, предполагая, что ее записал порожденный процесс, то могла бы возникнуть ситуация, в которой процесс-родитель прочитал бы собственную информацию, а процесс-ребенок не получил бы ничего. Более простой способ организации двунаправленной связи между родственными процессами заключается в использовании двух pip'ов. Модифицируйте программу из предыдущего примера для организации такой двусторонней связи, откомпилируйте ее и запустите на исполнение.

Необходимо отметить, что в некоторых UNIX-подобных системах (например, в Solaris2) реализованы полностью дуплексные pip'ы [11]. В таких системах для обоих файловых дескрипторов, ассоциированных с pip'ом, разрешены и операция чтения, и операция записи. Однако, такое поведение не характерно для pip'ов и не является переносимым.

12.Особенности поведения вызовов read() и write() для pip'а.

Системные вызовы read() и write() имеют определенные особенности поведения при работе с pipe, связанные с его ограниченным размером, задержками в передаче данных и возможностью блокирования обменивающихся информацией процессов. Организация запрета блокирования этих вызовов для pipe выходит за рамки нашего курса. **Будьте внимательны при написании программ, обменивающихся большими объемами информации через pipe.**

Помните, что за один раз из pip'a может прочитаться меньше информации, чем вы запрашивали, и за один раз в pipe может записаться меньше информации, чем вам хотелось бы. Проверяйте значения, возвращаемые вызовами!

Одна из особенностей поведения блокирующегося системного вызова read() связана с попыткой чтения из пустого pip'a. Если есть процессы, у которых этот pipe открыт для записи, то системный вызов блокируется и ждет появления информации. Если таких процессов нет, он вернет значение 0 без блокировки процесса. Эта особенность приводит к необходимости закрытия файлового дескриптора, ассоциированного с входным концом pip'a, в процессе, который будет использовать pipe для чтения (close(fd[1]) в процессе-ребенке в программе \\ИБ3\БОС\linux\05-2.c). Аналогичной особенностью поведения при отсутствии процессов, у которых pipe открыт для чтения, обладает и системный вызов write(), с чем связана необходимость закрытия файлового дескриптора, ассоциированного с выходным концом pip'a, в процессе, который будет использовать pipe для записи (**close(fd[0])** в процессе-родителе в той же программе).

Задача повышенной сложности: определите размер pipe для вашей операционной системы.

13.Понятие о FIFO. Использование системного вызова mknod для создания FIFO. Функция mkfifo.

Как мы выяснили, доступ к информации о расположении pip'a в операционной системе и его состоянии может быть осуществлен только через таблицу открытых файлов процесса, создавшего pipe, и через унаследованные от него таблицы открытых файлов процессов-потомков. Поэтому, изложенный выше механизм обмена информацией через pipe справедлив лишь для родственных процессов, имеющих общего прародителя, инициировавшего системный вызов pipe(), или для таких процессов и самого прародителя и не может использоваться для потокового общения с другими процессами. Откровенно говоря, в операционной системе UNIX существует возможность использования pip'a для взаимодействия других процессов, но ее реализация достаточно сложна и лежит далеко за пределами наших занятий.

Для организации потокового взаимодействия любых процессов в операционной системе UNIX применяется средство связи, получившее название FIFO (от First Input First Output) или именованный pipe. FIFO во всем подобен pip'у, за одним исключением: данные о расположении FIFO в адресном пространстве ядра и его состоянии процессы могут получать не через родственные связи, а через файловую систему. Для этого при создании именованного pip'a на диске заводится файл специального типа, обращаясь к которому процессы могут узнать интересующую их информацию. Для создания FIFO применяется системный вызов mknod() или существующая в некоторых версиях UNIX функция mkfifo(). Следует отметить, что при их работе не происходит действительного выделения области адресного пространства операционной системы под именованный pipe, а только заводится файл-метка, существование которого позволяет осуществить реальную организацию FIFO в памяти при его открытии с помощью уже известного нам системного вызова open(). После открытия именованный pipe ведет себя точно так же, как и неименованный. Для дальнейшей работы с ним применяются системные вызовы read(), write() и close(). Время существования FIFO в адресном пространстве ядра операционной системы, как и в случае pip'a, не может превышать времени жизни последнего из использующих его процессов. Когда все процессы, работающие с FIFO, закрывают все файловые дескрипторы, ассоциированные с ним, система освобождает ресурсы, выделенные под FIFO. Вся непрочитанная информация теряется. В то же время файл-метка остается жить на диске, и он может быть использован для новой реальной организации FIFO в дальнейшем.

Важно понимать, что файл типа FIFO не служит для размещения на диске информации, которая записывается в именованный pipe. Эта информация располагается внутри адресного пространства операционной системы, а файл является только меткой, создающей предпосылки

для ее размещения. **Не пытайтесь просмотреть содержимое этого файла с помощью Midnight Commander (mc) !!! Это приведет к его глубокому зависанию!**

14.Особенности поведения вызова open() при открытии FIFO.

Системные вызовы read() и write() при работе с FIFO имеют такие же особенности поведения, как и при работе с pip'ом. Системный вызов open() при открытии FIFO также ведет себя несколько иначе, чем при открытии других типов файлов, что связано с возможностью блокирования выполняющих его процессов. Если FIFO открывается только для чтения, и не задан флаг **O_NDELAY**, то процесс, осуществивший системный вызов, блокируется до тех пор, пока какой-либо другой процесс не откроет FIFO на запись. Если флаг **O_NDELAY** задан, то возвращается значение файлового дескриптора, ассоциированного с FIFO. Если FIFO открывается только для записи, и не задан флаг **O_NDELAY**, то процесс, осуществивший системный вызов, блокируется до тех пор, пока какой-либо другой процесс не откроет FIFO на чтение. Если флаг **O_NDELAY** задан, то констатируется возникновение ошибки и возвращается значение -1. Задание флага **O_NDELAY** в параметрах системного вызова open() приводит и к тому, что процессу, открывшему FIFO, запрещается блокировка при выполнении последующих операций чтения из этого потока данных и записи в него.

15.Прогон программы с FIFO в родственных процессах.

Для иллюстрации взаимодействия процессов через FIFO давайте рассмотрим программу, находящуюся в файле \\ИБ3\БОС\linux\05-4.c, откомпилируем ее и запустим на исполнение. В этой программе информацией между собой обмениваются процесс-родитель и процесс-ребенок. Обратим внимание, что повторный запуск этой программы приведет к ошибке при попытке создания FIFO, так как файл с заданным именем уже существует. Здесь нужно либо удалять его перед каждым прогоном программы с диска вручную , либо после первого запуска модифицировать исходный текст, исключив из него все, связанное с ситемным вызовом mknod(). С системным вызовом, предназначенным для удаления файла при работе процесса, мы познакомимся позже при изучении файловых систем.

16.Неработающий пример для связи процессов на различных компьютерах.

Если у вас есть возможность, найдите два компьютера, имеющих разделяемую файловую систему (например, смонтированную с помощью NFS), и запустите на них программы из предыдущего раздела так, чтобы каждая программа работала на своем компьютере, а FIFO создавалось на разделяемой файловой системе. Хотя оба процесса видят файл с типом FIFO, взаимодействия между ними не происходит, так как они функционируют в физически разных адресных пространствах и пытаются открыть FIFO внутри разных операционных систем.

Задание:

1. Измените программу из пункта 6 так, чтобы она читала записанную ранее в файл информацию и печатала ее на экране. Желательно удалить все лишние операторы.
2. Откомпилируйте и запустите на исполнение программу, исходный текст которой расположен в файле \\ИБ3\БОС\linux\05-2.c.(пункт 9)
3. Задача повышенной сложности: модифицируйте пример находящийся в файле \\ИБ3\БОС\ linux\05-3.c для связи между собой двух родственных процессов, исполняющих разные программы.
4. Для закрепления полученных знаний напишите на базе предыдущего примера (пункт 16) две программы, одна из которых пишет информацию в FIFO, а вторая - читает из него, так чтобы между ними не было ярко выраженных родственных связей (т.е. чтобы одна из них не была потомком другой).

Контрольные вопросы

1. Понятие о потоке ввода-вывода

2. Понятие о работе с файлами через системные вызовы и стандартную библиотеку ввода-вывода.
3. Понятие о файловом дескрипторе.
4. Открытие файла. Системный вызов open().
5. Системные вызовы close(), read(), write().
6. Понятие о pipe. Системный вызов pipe().
7. Организация связи через pipe между процессом-родителем и процессом-потомком. Наследование файловых дескрипторов при вызовах fork() и exec().
8. Особенности поведения вызовов read() и write() для pip&rsquoa.
9. Понятие о FIFO. Использование системного вызова mknod для создания FIFO. Функция mkfifo.
10. Особенности поведения вызова open() при открытии FIFO.

ЛАБОРАТОРНЫЕ РАБОТЫ №9. СРЕДСТВА SYSTEM V IPC. ОРГАНИЗАЦИЯ РАБОТЫ С РАЗДЕЛЯЕМОЙ ПАМЯТЬЮ В UNIX. ПОНЯТИЕ НИТЕЙ ИСПОЛНЕНИЯ (THREAD'OB).

Цели занятия

- Дать общее представление о System V IPC.
- Привить навыки работы с разделяемой памятью.
- Дать опыт работы с командами ipcs и ipcrm.
- Научить создавать и завершать threads в рамках одного процесса.
- Студент должен осознать разницу между созданием нового процесса и созданием нового thread'a.
- Студент должен понять необходимость синхронизации процессов и нитей исполнения при использовании разделяемой памяти.

Практические работы

- Прогон программ с использованием разделяемой памяти
- Работа с командами ipcs и ipcrm.
- Самостоятельное написание, компиляция и запуск программы для организации связи двух процессов через разделяемую память.
- Прогон программы с использованием двух нитей исполнения.
- Написание, компиляция и прогон программы с использованием трех нитей исполнения.
- Прогон программ, иллюстрирующих необходимость синхронизации процессов и нитей исполнения, использующих общую память.

1.Преимущества и недостатки потокового обмена данными.

Потоковые механизмы достаточно просты в реализации и удобны для использования, но обладают рядом существенных недостатков:

Операции чтения и записи не анализируют содержимое передаваемых данных. Процесс, прочитавший 20 байт из потока, не может сказать, были ли они записаны одним процессом или несколькими, записывались ли они за один раз или было, например, выполнено 4 операции записи по 5 байт. Данные в потоке никак не интерпретируются системой. Если требуется какая-либо интерпретация данных, то передающий и принимающий процесс должны заранее согласовать свои действия и уметь осуществлять ее самостоятельно.

Для передачи информации от одного процесса к другому требуется, как минимум, две операции копирования данных: первый раз - из адресного пространства передающего процесса в системный буфер, второй раз - из системного буфера в адресное пространство принимающего процесса.

Процессы, обменивающиеся информацией, должны одновременно существовать в вычислительной системе. Нельзя записать информацию в поток с помощью одного процесса, завершить его, а затем, через некоторое время, запустить другой процесс и прочитать записанную информацию.

2.Понятие о System V IPC.

Указанные выше недостатки потоков данных привели к разработке других механизмов передачи информации между процессами. Часть этих механизмов, впервые появившихся в UNIX System V и впоследствии перекочевавших оттуда практически во все современные версии операционной системы UNIX, получила общее название *System V IPC* (IPC - сокращение от InterProcess Communications). В группу System V IPC входят: очереди сообщений, разделяемая память и семафоры. Эти средства организации взаимодействия процессов связаны не только общностью происхождения, но и обладают схожим интерфейсом для выполнения схожих операций, например для выделения, и освобождения соответствующего ресурса в

системе. Их рассмотрение мы будем проводить в порядке от менее семантически нагруженных с точки зрения операционной системы к более семантически нагруженным. Иными словами, чем позже мы будем заниматься каким-либо механизмом из System V IPC, тем больше действий по интерпретации передаваемой информации приходится выполнять операционной системе при использовании этого механизма.

3.Пространство имен. Адресация в System V IPC. Функция ftok().

Все средства связи из System V IPC, как и уже рассмотренные нами pipe и FIFO, являются средствами связи с непрямой адресацией. Как мы установили ранее, для организации взаимодействия неродственных процессов с помощью средства связи с непрямой адресацией необходимо, чтобы это средство связи имело некоторое имя. Отсутствие имен у pip'ов позволяет процессам получать информацию о расположении pip'а в системе и его состоянии только через родственные связи. Наличие ассоциированного имени у FIFO - имени специализированного файла в файловой системе - позволяет неродственным процессам получать эту информацию через интерфейс файловой системы.

Множество всех возможных имен для объектов какого-либо вида принято называть пространством имен соответствующего вида объектов. Для FIFO пространством имен является множество всех допустимых имен файлов в файловой системе. Для всех объектов из System V IPC таким пространством имен является множество значений некоторого целочисленного типа данных - key_t. - ключа. Причем программисту не позволено напрямую присваивать значение ключа, это значение задается опосредовано: через комбинацию имени какого-либо файла, уже существующего в файловой системе, и небольшого целого числа - например, номера экземпляра средства связи.

Такой хитрый способ получения значения ключа связан с двумя соображениями:

Если разрешить программистам самим впрямую присваивать значение ключа для идентификации средств связи, то не исключено, что два различных программиста случайно воспользуются одним и тем же значением, не подозревая об этом. Тогда их процессы будут несанкционированно взаимодействовать через одно и то же средство коммуникации, что может привести к незапланированному поведению этих процессов. Поэтому основным компонентом значения ключа является преобразованное в числовое значение полное имя некоторого файла, доступ к которому на чтение разрешен процессу. Каждый программист имеет возможность использовать для этой цели свой специфический файл, например, исполняемый файл, связанный с одним из его взаимодействующих процессов. Следует отметить, что преобразование из текстового имени файла в число основывается на расположении указанного файла на жестком диске или ином физическом носителе. Поэтому для образования ключа следует применять файлы, не меняющие своего положения в течение времени организации взаимодействия процессов.

Второй компонент значения ключа используется для того, чтобы позволить программисту связать с одним и тем же именем файла более одного экземпляра каждого средства связи. В качестве такого компонента можно задавать порядковый номер соответствующего экземпляра.

Получение значения ключа из двух этих компонентов осуществляется функцией ftok().

Необходимо еще раз подчеркнуть три важных момента, связанных с использованием имени файла для получения ключа. Во-первых, необходимо указывать имя файла, который **уже существует в файловой системе** и для которого процесс **имеет право доступа на чтение** (не путайте с заданием имени файла при создании FIFO, где указывалось имя для **вновь создаваемого** специального файла). Во-вторых, указанный файл должен **сохранять свое положение на диске** до тех пор, пока все процессы, участвующие во взаимодействии, не получат ключ System V IPC. В-третьих, задание имени файла, как одной из компонент для получения ключа, ни в коем случае не означает, что информация, передаваемая с помощью ассоциированного средства связи, будет располагаться в этом файле. Информация будет

храниться **внутри адресного пространства операционной системы,** а заданное имя файла лишь позволяет различным процессам сгенерировать идентичные ключи.

4.Дескрипторы System V IPC.

Мы говорили, что информацию о потоках ввода-вывода, с которыми имеет дело текущий процесс, в частности о pip'ax и FIFO, операционная система хранит в таблице открытых файлов процесса. Системные вызовы, осуществляющие операции над потоком, используют в качестве параметра индекс элемента таблицы открытых файлов, соответствующего потоку, - файловый дескриптор. Использование файловых дескрипторов для идентификации потоков внутри процесса позволяет применять к ним уже существующий интерфейс для работы с файлами, но в то же время приводит к автоматическому закрытию потоков при завершении процесса. Этим в частности, объясняется один из недостатков потоковой передачи информации.

При реализации компонент System V IPC была принята другая концепция. Ядро операционной системы хранит информацию обо всех средствах System V IPC, используемых в системе, вне контекста пользовательских процессов. При создании нового средства связи или получении доступа к уже существующему процесс получает неотрицательное целое число - *дескриптор (идентификатор) этого средства связи*, которое однозначно идентифицирует его во всей вычислительной системе. Этот дескриптор должен передаваться в качестве параметра всем системным вызовам, осуществляющим дальнейшие операции над соответствующим средством System V IPC.

Подобная концепция позволяет устранить один из самых существенных недостатков, присущих потоковым средствам связи - требование об одновременном существовании взаимодействующих процессов, но в то же время требует повышенной осторожности для того, чтобы процесс, получающий информацию, не принял взамен новых старые, уже "протухшие" данные, по забывчивости оставленные в механизме коммуникации.

5.Разделяемая память в UNIX. Системные вызовы shmget(), shmat(), shmdt().

С точки зрения операционной системы, наименее семантически нагруженным средством System V IPC является разделяемая память (shared memory). Операционная система может позволить нескольким процессам совместно использовать некоторую область адресного пространства.

Все средства связи System V IPC требуют предварительных инициализирующих действий (своего создания) для организации взаимодействия процессов.

Для создания области разделяемой памяти с определенным ключом или доступа по ключу к уже существующей области применяется системный вызов shmget(). Существует два варианта его использования для создания новой области разделяемой памяти.

Стандартный способ. В качестве значения ключа системному вызову поставляется значение, сформированное функцией ftok() для некоторого имени файла и номера экземпляра области разделяемой памяти. В качестве флагов поставляется комбинация прав доступа к создаваемому сегменту и флага **IPC_CREAT**. Если сегмент для данного ключа еще не существует, то система будет пытаться создать его с указанными правами доступа. Если же вдруг он уже существовал, то мы просто получим его дескриптор. Возможно добавление к этой комбинации флагов флага **IPC_EXCL**. Этот флаг гарантирует нормальное завершение системного вызова только в том случае, если сегмент действительно был создан (т. е. ранее он не существовал), если же сегмент существовал, то системный вызов завершится с ошибкой, и значение системной переменной errno, описанной в файле **errno.h**, будет установлено в **EEXIST**.

Нестандартный способ. В качестве значения ключа указывается специальное значение **IPC_PRIVATE**. Использование значения I**PC_PRIVATE** всегда приводит к попытке создания нового сегмента разделяемой памяти с заданными правами доступа и с ключом, который не совпадает со значением ключа ни одного из уже существующих сегментов и который не может

быть получен с помощью функции **ftok()** ни при одной комбинации ее параметров. Наличие флагов **IPC_CREAT и IPC_EXCL** в этом случае игнорируется.

Доступ к созданной области разделяемой памяти в дальнейшем обеспечивается ее дескриптором, который вернет системный вызов shmget(). Доступ к уже существующей области также может быть осуществлен двумя способами:

Если мы знаем ее ключ, то, использовав вызов shmget(), мы можем получить ее дескриптор. В этом случае нельзя использовать в качестве составной части флагов флаг **IPC_EXCL**, а значение ключа, естественно, не может быть **IPC_PRIVATE**. Права доступа игнорируются, а размер области должен совпадать с размером, указанным при ее создании.

Либо мы можем воспользоваться тем, что дескриптор System **V IPC** действителен в рамках всей операционной системы, и передать его значение от процесса, создавшего разделяемую память, каким-нибудь образом текущему процессу. Отметим, что при создании разделяемой памяти с помощью значения **IPC_PRIVATE** - это единственно возможный способ.

После получения дескриптора необходимо включить область разделяемой памяти в адресное пространство текущего процесса. Это осуществляется с помощью системного вызова shmat(). При нормальном завершении он вернет адрес разделяемой памяти в адресном пространстве текущего процесса. Дальнейший доступ к этой памяти осуществляется с помощью обычных средств языка программирования. После окончания использования разделяемой памяти процесс может уменьшить размер своего адресного пространства, исключив из него эту область с помощью системного вызова shmdt(). Отметим, что в качестве своего параметра системный вызов shmdt() требует адрес начала области разделяемой памяти в адресном пространстве процесса, т. е. значение, которое вернул системный вызов shmat(), поэтому данное значение следует сохранять на протяжении всего времени использования разделяемой памяти.

6.Прогон программ с использованием разделяемой памяти.

Для иллюстрации использования разделяемой памяти давайте рассмотрим две взаимодействующие программы, расположенные в файлах \\ИБ3\БОС\linux\06-1a.c и \\ИБ3\ БОС\linux\06-1b.c. Обе программы очень похожи друг на друга и используют разделяемую память для хранения числа запусков каждой из программ и их суммы. В разделяемой памяти размещается массив из трех целых чисел. Первый элемент массива используется как счетчик для программы 1, второй элемент - для программы 2, третий элемент - для обеих программ суммарно. Дополнительный нюанс в программах возникает из-за необходимости инициализации элементов массива при создании разделяемой памяти. Для этого нам нужно, чтобы программы могли различать случай, когда они создали ее и случай, когда она уже существовала. Мы добиваемся различия, используя вначале системный вызов shmget() с флагами **IPC_CREAT и IPC_EXCL**. Если вызов завершается нормально, то мы создали разделяемую память. Если вызов завершается с констатацией ошибки и значение переменной errno равняется **EEXIST**, то, значит, разделяемая память уже существует, и мы можем получить ее **IPC** дескриптор, применяя тот же самый вызов с нулевым значением флагов. Откомпилируйте программы и запустите их несколько раз. Проанализируйте полученные результаты.

7.Команды ipc и ipcrm.

Как мы видели из предыдущего примера, созданная область разделяемой памяти сохраняется в операционной системе даже тогда, когда нет ни одного процесса, включающего ее в свое адресное пространство. С одной стороны, это имеет определенные преимущества, не требуя одновременного существования взаимодействующих процессов, с другой стороны может причинять существенные неудобства. Допустим, что предыдущие программы мы хотим использовать таким образом, чтобы подсчитывать количество запусков в течение одного, текущего, сеанса работы в системе. Однако в созданном сегменте разделяемой памяти остается

информация от предыдущего сеанса, и программы будут выдавать общее количество запусков за все время работы с момента загрузки операционной системы. Можно было бы создавать для нового сеанса новый сегмент разделяемой памяти, но количество ресурсов в системе не безгранично. Нас спасает то, что существуют способы удалять неиспользуемые ресурсы System V IPC как с помощью команд операционной системы, так и с помощью системных вызовов. Все средства System V IPC требуют определенных действий для освобождения занимаемых ресурсов после окончания взаимодействия процессов. Для того, чтобы удалять ресурсы System V IPC из командной строки, нам понадобятся две команды ipcs и ipcrm. Команда ipcs выдает информацию обо всех средствах System V IPC, существующих в системе, для которых пользователь обладает правами на чтение: областях разделяемой памяти, семафорах и очередях сообщений. Из всего многообразия выводимой информации нас будут интересовать только IPC идентификаторы для средств, созданных вами. Эти идентификаторы будут использоваться в команде ipcrm, позволяющей удалить необходимый ресурс из системы. Для удаления сегмента разделяемой памяти эта команда имеет вид:

ipcrm shm <IPC идентификатор>

Удалите созданный вами сегмент разделяемой памяти из операционной системы, используя эти команды.

Если поведение ваших программ, использующих средства System V IPC, базируется на предположении, что эти средства были созданы при их работе, не забывайте перед их запуском удалять уже существовавшие ресурсы.

8.Использование системного вызова shmctl() для освобождения ресурса.

Для той же цели - удалить область разделяемой памяти из системы - можно воспользоваться и системным вызовом shmctl(). Этот системный вызов позволяет полностью ликвидировать область разделяемой памяти в операционной системе по заданному дескриптору средства IPC, если, конечно, у вас хватает для этого полномочий. Кроме этого, системный вызов shmctl() позволяет выполнять и другие действия над сегментом разделяемой памяти, но их изучение лежит за пределами нашего курса.

9.Разделяемая память и системные вызовы fork(), exec() и функция exit().

Важным вопросом является поведение сегментов разделяемой памяти при выполнении процессом системных вызовов fork(), exec() и функции exit().

При выполнении системного вызова fork() все области разделяемой памяти, размещенные в адресном пространстве процесса, наследуются порожденным процессом.

При выполнении системных вызовов exec() и функции exit() все области разделяемой памяти, размещенные в адресном пространстве процесса, исключаются из его адресного пространства, но продолжают существовать в операционной системе.

10.Понятие о нити исполнения (thread) в UNIX. Идентификатор нити исполнения. Функция pthread_self().

Во многих современных операционных системах существует расширенная реализация понятия процесс, когда процесс представляет собой совокупность выделенных ему ресурсов и набора нитей исполнения. Нити процесса разделяют его программный код, глобальные переменные и системные ресурсы, но каждая нить имеет свой собственный программный счетчик, свое содержимое регистров и свой собственный стек. Поскольку глобальные переменные у нитей исполнения являются общими, то они могут использовать их, как элементы разделяемой памяти, не прибегая к механизму, описанному выше.

В различных версиях операционной системы UNIX существуют различные интерфейсы, обеспечивающие работу с нитями исполнения. Мы с вами кратко ознакомимся с некоторыми функциями, позволяющими разделить процесс на thread'ы и управлять их поведением, соответствующими стандарту POSIX. Нити исполнения, удовлетворяющие стандарту POSIX, принято называть POSIX thread'ами или кратко pthread'ами.

К сожалению, операционная система Linux не полностью поддерживает нити исполнения на уровне ядра системы. При создании нового thread'a запускается новый традиционный процесс, разделяющий с родительским традиционным процессом его ресурсы, программный код и данные, расположенные вне стека, т.е. фактически действительно создается новый thread, но ядро не умеет определять, что эти thread'ы являются составными частями одного целого. Это знает только специальный процесс-координатор, работающий на пользовательском уровне и стартующий при первом вызове функций, обеспечивающих POSIX интерфейс для нитей исполнения. Поэтому мы сможем наблюдать не все преимущества использования нитей исполнения (в частности, ускорить решение задачи с их помощью вряд ли получится), но даже в этом случае, thread'ы можно использовать как очень удобный способ для создания процессов с общими ресурсами, программным кодом и разделяемой памятью.

Каждая нить исполнения, как и процесс, имеет в системе свой собственный уникальный номер - идентификатор thread'a. Поскольку традиционный процесс в концепции нитей исполнения трактуется как процесс, содержащий единственную нить исполнения, то мы можем узнать идентификатор этой нити и для любого обычного процесса. Для этого используется функция pthread_self(). Нить исполнения создаваемую при рождении нового процесса принято называть *начальной* или *главной нитью исполнения этого процесса*.

11.Создание и завершение thread'a. Функции pthread_create(), pthread_exit(), pthread_join().

Нити исполнения, как и традиционные процессы, могут порождать нити-потомки, правда, только внутри своего процесса. Каждый будущий thread внутри программы должен представлять собой функцию с прототипом

void *thread(void *arg)

Параметр **arg** передается этой функции при создании thread'a и может, до некоторой степени, рассматриваться как аналог параметров функции **main()**. Возвращаемое функцией значение может интерпретироваться как аналог информации, которую родительский процесс может получить после завершения процесса-ребенка. Для создания новой нити исполнения применяется функция pthread_create(). Мы не будем рассматривать ее в полном объеме, так как детальное изучение программирования с использованием thread'ов не является целью данного курса. Важным отличием этой функции от большинства других системных вызовов и функций является то, что в случае неудачного завершения она возвращает не отрицательное, а положительное значение, которое определяет код ошибки, описанный в файле **errno.h**. Значение системной переменной **errno** при этом не устанавливается. Результатом выполнения этой функции является появление в системе новой нити исполнения, которая будет выполнять функцию, ассоциированную со thread'ом, передав ей специфицированный параметр, параллельно с уже существовавшими нитями исполнения процесса.

Созданный thread может завершить свою деятельность тремя способами

С помощью выполнения функции pthread_exit(). Функция никогда не возвращается в вызвавшую ее нить исполнения. Объект, на который указывает параметр этой функции, может быть изучен в другой нити исполнения, например, в породившей завершившийся thread. Этот параметр, следовательно, должен указывать на объект, не являющийся локальным для завершившегося thread'a, например, на статическую переменную.

С помощью возврата из функции, ассоциированной с нитью исполнения. Объект, на который указывает адрес, возвращаемый функцией, как и в предыдущем случае, может быть изучен в другой нити исполнения, например, в породившей завершившийся thread, и должен указывать на объект, не являющийся локальным для завершившегося thread'a.

Если в процессе выполняется возврат из функции **main()** или где-либо в процессе (в любой нити исполнения) осуществляется вызов функции exit(), то это приводит к завершению всех thread'ов процесса.

Одним из вариантов получения адреса, возвращаемого завершившимся thread'ом, с одновременным ожиданием его завершения является использование функции pthread_join(). Нить исполнения, вызвавшую эту функцию, переходит в состояние ожидание до завершения заданного thread'a. Функция позволяет также получить указатель, который вернул завершившийся thread в операционную систему.

12.Прогон программы с использованием двух нитей исполнения.

Для иллюстрации вышесказанного давайте рассмотрим программу \\ИБ3\БОС\linux\06-2.c, в которой работают две нити исполнения. При работе редактора связей необходимо явно подключить библиотеку функции для работы с pthread'ами, которая не подключается автоматически. Это делается с помощью добавления к команде компиляции и редактирования связей параметра -lpthread - подключить библиотеку pthread. Откомпилируйте эту программу и запустите на исполнение. Обратите внимание на отличие результатов этой программы от похожей программы, иллюстрировавшей создание нового процесса. Программа, создававшая новый процесс, печатала дважды одинаковые значения для переменной a, так как адресные пространства различных процессов независимы, и каждый процесс прибавлял 1 к своей собственной переменной a. Рассматриваемая программа печатает два разных значения, так как переменная a является разделяемой, и каждый thread прибавляет 1 к одной и той же переменной.

13.Необходимость синхронизации процессов и нитей исполнения, использующих общую память.

Все рассмотренные на этом семинаре примеры являются не совсем корректными. В большинстве случаев они работают правильно, однако возможны ситуации, когда совместная деятельность этих процессов или нитей исполнения приводит к неверным и неожиданным результатом. Это связано с тем, что любые неатомарные операции, связанные с изменением содержимого разделяемой памяти, представляют собой критическую секцию процесса или нити исполнения. Вернемся к рассмотрению программ \\ИБ3\БОС\linux\06-1a.c и \\ИБ3\БОС\linux\06-1b.c. При одновременном существовании 2-х процессов в операционной системе может возникнуть следующая последовательность выполнения операций во времени:

...
Процесс 1: array[0] += 1;
Процесс 2: array[1] += 1;
Процесс 1: array[2] += 1;
Процесс 1: printf("Program 1 was spawn %d times, program 2 - %d times, total - %d times\n", array[0], array[1], array[2]);
...

Тогда печать будет давать неправильные результаты. Естественно, что воспроизвести подобную последовательность действий практически нереально. Мы не сможем подобрать необходимые времена старта процессов и степень загруженности вычислительной системы. Но мы можем смоделировать эту ситуацию, добавив в обеих программах достаточно длительные пустые циклы перед оператором **array[2] += 1**; Это проделано в программах \\ИБ3\БОС\linux\06-3a.c и \\ИБ3\БОС\linux\06-3b.c. Откомпилируем их, запустим любую из них один раз для создания и инициализации разделяемой памяти. Затем запустим другую и, пока она находится в цикле, запустим, например, с другого виртуального терминала, снова первую. Мы получим неожиданный результат. Как видим, для написания корректно работающих программ необходимо обеспечивать взаимоисключение при работе с разделяемой памятью и, может быть, взаимную очередность доступа к ней. Это можно сделать с помощью рассмотренных на лекции алгоритмов синхронизации, например, алгоритма Петерсона или алгоритма булочной.

Задача повышенной сложности: модифицируйте программы \\ИБ3\БОС\linux\06-3a.c и \\ИБ3\БОС\linux\06-3b.c для корректной работы с помощью алгоритма Петерсона.

Задание:

1. Для закрепления полученных знаний напишите две программы, осуществляющие взаимодействие через разделяемую память. Первая программа должна создавать сегмент разделяемой памяти и копировать туда свой собственный исходный текст, вторая программа должна брать оттуда этот текст, печатать его на экране и удалять сегмент разделяемой памяти из системы.
2. Модифицируйте программу из пункта 13, добавив к ней третью нить исполнения.

Контрольные вопросы

1. Преимущества и недостатки потокового обмена данными.
2. Понятие о System V IPC.
3. Пространство имен. Адресация в System V IPC. Функция ftok().
4. Дескрипторы System V IPC.
5. Разделяемая память в UNIX. Системные вызовы shmget(), shmat(), shmdt().
6. Команды ipc и ipcrm.
7. Использование системного вызова shmctl() для освобождения ресурса.
8. Разделяемая память и системные вызовы fork(), exec() и функция exit().
9. Понятие о нити исполнения (thread) в UNIX. Идентификатор нити исполнения. Функция pthread_self().
10. Создание и завершение thread'a. Функции pthread_create(), pthread_exit(), pthread_join().
11. Необходимость синхронизации процессов и нитей исполнения, использующих общую память.

ЛАБОРАТОРНАЯ РАБОТА №10. СЕМАФОРЫ В UNIX КАК СРЕДСТВО СИНХРОНИЗАЦИИ ПРОЦЕССОВ.

Цели занятия

- Дать навыки работы с семафорами в UNIX. Хотя работа с семафорами является достаточно простой идеологически, эта тема вызывает у студентов серьезные затруднения из-за сложности семантики системного вызова semop().
- Дать представление о существовании POSIX семафоров.

Практические работы

- Прогон примера с использованием семафора.
- Изменение предыдущего примера.
- Написание, компиляция и прогон программы с организацией взаимоисключения с помощью семафоров для 2-х процессов, взаимодействующих через разделяемую память.
- Написание, компиляция и прогон программы с организацией взаимной очередности с помощью семафоров для 2-х процессов, взаимодействующих через pipe.

1.Семафоры в UNIX. Отличие операций над UNIX семафорами от классических операций.

Как упоминалось на лекции, одним из первых механизмов, предложенных для синхронизации поведения процессов, стали семафоры, концепцию которых описал Дейкстра (Dijkstra) в 1965 году. При разработке средств System V IPC семафоры вошли в их состав как неотъемлемая часть. Следует отметить, что набор операций над семафорами System V IPC отличается от классического набора операций **{P, V },** предложенного Дейкстрой. Он насчитывает три операции:

- **A(S, n)** - увеличить значение семафора S на величину n;
- **D(S, n)** - пока значение семафора S < n процесс блокируется. Далее S = S - n;

Z(S) - процесс блокируется до тех пор, пока значение семафора S не станет равным 0.

Изначально все IPC семафоры инициируются нулевым значением.

Легко видеть, что классической операции **P(S**) соответствует операция **D(S**,**1)**, а классической операции **V(S)** соответствует операция **A(S,1)**. Аналогом ненулевой инициализации Дейкстровских семафоров значением n может служить выполнение операции **A(S,n)** сразу после создания семафора **S**, с обеспечением атомарности создания семафора и ее выполнения посредством другого семафора. Мы показали, что классические семафоры реализуются через семафоры **System V IPC**. Обратное не является верным. Используя операции **P(S**) и **V(S**), мы не сумеем реализовать операцию **Z(S)**.

Поскольку IPC семафоры являются составной частью средств System V IPC, то для них верно все, что говорилось об этих средствах в целом на предыдущем семинаре. IPC семафоры являются средством связи с непрямой адресацией, требуют инициализации для организации взаимодействия процессов и специальных действия для освобождения системных ресурсов по его окончании. Пространством имен IPC семафоров является множество значений ключа, генерируемых с помощью функции ftok(). Для совершения операций над семафорами системным вызовам в качестве параметра передаются IPC дескрипторы семафоров, однозначно идентифицирующих их во всей вычислительной системе, а вся информация о семафорах располагается в адресном пространстве ядра операционной системы. Это позволяет организовывать через семафоры взаимодействие процессов, даже не находящихся в системе одновременно.

2.Создание массива семафоров или доступ к уже существующему. Системный вызов semget().

В целях экономии системных ресурсов операционная система UNIX позволяет создавать не по одному семафору для каждого конкретного значения ключа, а связывать с ключом целый массив семафоров (в Linux - до 500 семафоров в массиве, хотя это количество может быть уменьшено системным администратором). Для создания массива семафоров, ассоциированного с определенным ключом, или доступа по ключу к уже существующему массиву используется системный вызов semget(), являющийся аналогом системного вызова shmget() для разделяемой памяти, который возвращает значение IPC дескриптора для этого массива. При этом существуют те же способы создания и доступа, что и для разделяемой памяти. Вновь созданные семафоры инициируются нулевым значением.

3.Выполнение операций над семафорами. Системный вызов semop().

Для выполнения операций **A, D и Z** над семафорами из массива используется системный вызов semop(), обладающий довольно сложной семантикой. Разработчики System V IPC явно перегрузили этот вызов, применяя его не только для выполнения всех трех операций, но еще и для нескольких семафоров в массиве IPC семафоров одновременно. Для правильного использования этого вызова необходимо выполнить следующие действия:

Определиться, для каких семафоров из массива вы хотите выполнить операции. Необходимо иметь в виду, что все операции реально совершаются только перед успешным возвращением из системного вызова, т.е. если вы хотите выполнить операции **A(S1,5**) и **Z(S2**) в одном вызове и оказалось, что **S2 != 0**, то значение семафора **S1** не будет изменено до тех пор, пока значение S2 не станет равным 0. Порядок выполнения операций в случае, когда процесс не переходит в состояние ожидание, не определен. Так, например, при одновременном выполнении операций A**(S1,1)** и **D(S2,1)** в случае **S2 > 1** неизвестно, что выполнится раньше - уменьшение значения семафора **S2** или увеличение значения семафора **S1**. Если порядок является для вас существенным, лучше применить несколько вызовов вместо одного.

После того как вы определились с количеством семафоров и совершаемыми операциями, необходимо завести в программе массив из элементов типа **struct sembuf** с размерностью равной определенному количеству семафоров (если операция совершается только над одним семафором можно, естественно, обойтись просто переменной). Каждый элемент этого массива будет соответствовать операции над одним семафором.

Заполнить элементы массива. В поле **sem_flg** каждого элемента занести значение **0** (с другими значениями флагов мы на семинарах работать не будем). В поля **sem_num** и sem_op занести номера семафоров в массиве IPC семафоров и соответствующие коды операций. Семафоры нумеруются, начиная с 0. Если у вас в массиве всего один семафор, то он будет иметь номер 0. Операции кодируются так: для выполнения операции **A(S,n**) значение поля **sem_op** должно быть равно **n**. для выполнения операции **D(S,n)** значение поля **sem_op** должно быть равно **-n**. для выполнения операции **Z(S)** значение поля **sem_op** должно быть равно **0**.

В качестве второго параметра системного вызова **semop()** указать адрес заполненного массива, а в качестве третьего параметра - ранее определенное количество семафоров, над которыми совершаются операции.

4.Прогон примера с использованием семафора.

Для иллюстрации вышесказанного давайте рассмотрим простейшие программы, синхронизирующие свои действия с помощью семафоров (файлы \\ИБ3\БОС\linux\/08-1a.c и \\ ИБ3\БОС\linux\/08-1b.c). Первая программа выполняет над семафором S операцию D(S,1), вторая программа выполняет над тем же семафором операцию **A(S,1)**. Если семафор не существует в системе, любая программа создает его перед выполнением операции. Поскольку при создании семафор всегда инициируется 0, то программа 1 может работать без блокировки только после запуска программы 2 .

5.Удаление набора семафоров из системы с помощью команды ipcrm или системного вызова semctl().

Как мы говорили и видели из примеров, массив семафоров может продолжать существовать в системе и после завершения использовавших его процессов, а семафоры будут сохранять свое значение. Это способно привести к неправильному поведению программ, предполагающих, что семафоры были только что созданы и, следовательно, имеют нулевое значение. Необходимо удалять семафоры из системы перед запуском таких программ или перед их завершением. Для удаления семафоров можно воспользоваться командами ipcs и ipcrm. Команда ipcrm в этом случае должна иметь вид

ipcrm sem <IPC идентификатор>

Для этой же цели мы можем применять системный вызов semctl(), который умеет выполнять и другие операции над массивом семафоров, но их рассмотрение выходит за рамки нашего курса.

6.Написание, компиляция и прогон программы с организацией взаимоисключения с помощью семафоров для 2-х процессов, взаимодействующих через разделяемую память.

На прошлом занятии мы установили, что любые неатомарные операции, связанные с изменением содержимого разделяемой памяти, представляют собой критическую секцию процесса или нити исполнения.

7.Понятие о POSIX семафорах.

В стандарте POSIX вводятся другие семафоры, полностью аналогичные семафорам Дейкстры. Для инициализации значения таких семафоров применяется функция **sem_init(),** аналогом операции **P** служит функция **sem_wait()**, а аналогом операции **V** - функция **sem_post().** К сожалению, в Linux такие семафоры реализованы только для нитей исполнения внутри одного процесса, и поэтому подробно мы на них останавливаться не будем.

Задание:

1. Измените предыдущие программы (пункт 4), так чтобы первая программа могла работать без блокировки после не менее 5 запусков второй программы.

2.Модифицируйте программы \\ИБ3\БОС\linux\06-3a.c и \\ИБ3\БОС\linux\06-3b.c, которые иллюстрировали некорректную работу через разделяемую память, обеспечив с помощью семафоров взаимоисключения для их правильной работы.

3. Организуйте двустороннюю поочередную связь процесса-родителя и процесса-ребенка через pipe, используя для синхронизации семафоры, модифицировав программу \\ИБ3\БОС\linux\05-3.c.

Контрольные вопросы

1. Семафоры в UNIX. Отличие операций над UNIX семафорами от классических операций.
2. Создание массива семафоров или доступ к уже существующему. Системный вызов semget().
3. Выполнение операций над семафорами. Системный вызов semop().
4. Удаление набора семафоров из системы с помощью команды ipcrm или системного вызова semctl().
5. Понятие о POSIX семафорах.

ЛАБОРАТОРНАЯ РАБОТА № 11. ОЧЕРЕДИ СООБЩЕНИЙ В UNIX.

Цели занятия

- Дать понятие об очередях сообщений в UNIX.
- Привить навыки работы с системными вызовами msgget(), msgsnd(), msgrcv(), msgctl(),
- Студенты должны понять, что сообщения не обязаны быть текстовыми.
- Дать понятие о мультиплексировании и модели клиент-сервер.

Практические работы

- Прогон примера с однонаправленной передачей текстовой информации.
- Модификация предыдущего примера для передачи числовой информации.
- Написание, компиляция и прогон программ для осуществления двусторонней связи через одну очередь сообщений.
- Написание, компиляция и прогон программ клиент и сервер.

1.Сообщения как средства связи и средства синхронизации процессов.

Третьим и последним, наиболее семантически нагруженным средством, входящим в System V IPC, являются *очереди сообщений.* На лекции мы говорили о модели сообщений, как о способе взаимодействия процессов через линии связи, в котором на передаваемую информацию накладывается определенная структура так, что процесс, принимающий данные, может четко определить, где заканчивается одна порция информации и начинается другая. Такая модель позволяет использовать одну и ту же линию связи для передачи данных в двух направлениях между несколькими процессами. Мы также говорили о возможности использования сообщений с встроенными механизмами взаимоисключения и блокировки при чтении из пустого буфера и записи в переполненный буфер для организации синхронизации процессов.

2.Очереди сообщений в UNIX как составная часть System V IPC.

Так как очереди сообщений входят в состав средств System V IPC, то для них верно все, что говорилось ранее об этих средствах в целом и уже знакомо нам. Очереди сообщений, как и семафоры, как и разделяемая память, являются средством связи с непрямой адресацией, требуют инициализации для организации взаимодействия процессов и специальных действий для освобождения системных ресурсов по окончании взаимодействия. Пространством имен очередей сообщений является то же самое множество значений ключа, генерируемых с помощью функции ftok(). Для выполнения примитивов **send и receive** соответствующим системным вызовам в качестве параметра передаются IPC дескрипторы очередей сообщений, однозначно идентифицирующих их во всей вычислительной системе.

Очереди сообщений располагаются в адресном пространстве ядра операционной системы в виде однонаправленных списков и имеют ограничение по объему информации, хранимой в каждой очереди. Каждый элемент списка представляет собой отдельное сообщение. Сообщения имеют атрибут, называемый типом сообщения. Выборка сообщений из очереди (выполнение примитива receive) может осуществляться тремя способами:

- В порядке FIFO, независимо от типа сообщения.
- В порядке FIFO для сообщений конкретного типа.

Первым выбирается сообщение с минимальным типом, не превышающим некоторого заданного значения, пришедшее ранее всех других сообщений с тем же типом.

Реализация примитивов **send и receive** обеспечивает скрытое от пользователя взаимоисключение во время помещения сообщения в очередь или его получении из очереди, а также блокировку процесса при попытке выполнить примитив receive над пустой очередью или очередью, в которой отсутствуют сообщения запрошенного типа, или при попытке выполнить примитив send для очереди, в которой нет свободного места.

Очереди сообщений, как и другие средства System V IPC, позволяют организовывать взаимодействие процессов, не находящихся одновременно в вычислительной системе.

3.Создание очереди сообщений или доступ к уже существующей. Системный вызов msgget().

Для создания очереди сообщений, ассоциированной с определенным ключом, или доступа по ключу к уже существующей очереди используется системный вызов msgget(), являющийся аналогом системных вызовов shmget() для разделяемой памяти и semget() для массива семафоров, который возвращает значение IPC дескриптора для этой очереди. При этом существуют те же способы создания и доступа, что и для разделяемой памяти или семафоров.

4.Реализация примитивов send и receive. Системные вызовы msgsnd() и msgrcv().

Для выполнения примитива **send** служит системный вызов **msgsnd()**, копирующий пользовательское сообщение в очередь сообщений, заданную своим IPC дескриптором. При изучении описания этого вызова обратите особое внимание на следующие моменты:

Тип данных **struct msgbuf** не является типом данных для пользовательских сообщений, а представляет собой лишь шаблон для создания таких типов. Пользователь сам должен создать структуру для своих сообщений, в которой первым полем обязана быть переменная типа **long**, содержащая положительное значение типа сообщения.

В качестве третьего параметра - длины сообщения - указывается не вся длина структуры данных, соответствующей сообщению, а только длина полезной информации, т. е. информации, располагающейся в структуре данных после типа сообщения. Это значение может быть и равным 0 в случае, когда вся полезная информация заключается в самом факте наличия сообщения (сообщение используется как сигнальное средство связи).

На наших занятиях мы, как правило, будем использовать нулевое значение флага системного вызова, которое приводит к блокировке процесса при отсутствии достаточного свободного места в очереди сообщений.

Примитив **receive** реализуется системным вызовом **msgrcv().** При изучении описания этого вызова обратите особое внимание на следующие моменты:

Тип данных struct msgbuf, как и для вызова msgsnd(), является лишь шаблоном для пользовательского типа данных.

Способ выбора сообщения задается нулевым, положительным или отрицательным значением параметра **type.** Точное значение типа выбранного сообщения можно определить из соответствующего поля структуры, в которую системный вызов скопирует сообщение.

Системный вызов возвращает длину только полезной части скопированной информации, т. е. информации, расположенной в структуре после поля типа сообщения.

Выбранное сообщение удаляется из очереди сообщений.

В качестве параметра **length** указывается максимальная длина полезной части информации, которая может быть размещена в структуре, адресованной параметром ptr.

На наших занятиях мы будем, как правило, пользоваться нулевым значением флагов для системного вызова, которое приводит к блокировке процесса в случае отсутствия в очереди сообщений с запрошенным типом и к ошибочной ситуации в случае, когда длина информативной части выбранного сообщения превышает длину, специфицированную в параметре **length.**

Максимально возможная длина информативной части сообщения в операционной системе Linux составляет 4080 байт и может быть уменьшена при генерации системы. Текущее значение максимальной длины можно определить с помощью команды

ipcs -l

5.Удаление очереди сообщений из системы с помощью команды ipcrm или системного вызова msgctl().

После завершения процессов, использовавших очередь сообщений, она не удаляется из системы автоматически, а продолжает сохраняться в системе вместе со всеми невостребованными сообщениями до тех пор, пока не будет выполнена специальная команда или специальный системный вызов. Для удаления очереди сообщений можно воспользоваться уже знакомой нам командой ipcrm, которая в этом случае примет вид:

ipcrm msg <IPC идентификатор>

Для получения IPC идентификатора очереди сообщений примените команду ipcs. Можно удалить очередь сообщений и с помощью системного вызова msgctl(). Этот вызов умеет выполнять и другие операции над очередью сообщений, но их рассмотрение лежит вне нашего курса. Если какой-либо процесс находился в состоянии *ожидание* при выполнении системного вызова msgrcv() или msgsnd() для удаляемой очереди, то он будет разблокирован, и системный вызов констатирует наличие ошибочной ситуации.

6.Прогон примера с однонаправленной передачей текстовой информации

Для иллюстрации всего вышесказанного давайте рассмотрим простые программы \\ИБ3\БОС\linux\09-1a.c и \\ИБ3\БОС\linux\09-1b.c. Первая из этих программ посылает пять текстовых сообщений с типом 1 и одно сообщение нулевой длины с типом 255 второй программе. Вторая программа в цикле принимает сообщения любого типа в порядке FIFO и печатает их содержимое до тех пор, пока не получит сообщение с типом 255. Сообщение с типом 255 служит для нее сигналом к завершению работы и ликвидации очереди сообщений. Если перед запуском любой из программ очередь сообщений еще отсутствовала в системе, то программа создаст ее.

Обратите внимание на использование сообщения с типом 255 в качестве сигнала прекращения работы второго процесса. Это сообщение имеет нулевую длину, так как его информативность исчерпывается самим фактом наличия сообщения.

7.Модификация предыдущего примера для передачи числовой информации.

В описании системных вызовов msgsnd() и msgrcv() говорится о том, что передаваемая информации не обязательно должна представлять собой текст. Мы можем воспользоваться очередями сообщений для передачи данных любого вида. При передаче разнородной информации целесообразно информативную часть объединять внутри сообщения в отдельную структуру:

```
struct mymsgbuf {
long mtype;
struct {
short sinfo;
float finfo;
} info;
} mybuf;
```

для правильного вычисления длины информативной части. В некоторых вычислительных системах числовые данные размещаются в памяти с выравниванием на определенные адреса (например, на адреса кратные 4). Поэтому реальный размер памяти, необходимой для размещения нескольких числовых данных, может оказаться больше суммы длин этих данных, т. е. в нашем случае, вообще говоря, **sizeof(info)>=sizeof(short)+sizeof(float)**. Для полной передачи информативной части сообщения в качестве длины нужно указывать не сумму длин полей, а полную длину структуры. Модифицируйте предыдущие программы для передачи нетекстовых сообщений.

8.Написание, компиляция и прогон программ для осуществления двусторонней связи через одну очередь сообщений.

Наличие у сообщений типов позволяет организовать двустороннюю связь между процессами через одну и ту же очередь сообщений. Процесс 1 может посылать процессу 2 сообщения с типом 1, а получать от него сообщения с типом 2. При этом для выборки сообщений в обоих процессах следует пользоваться вторым способом выбора.

9.Понятие мультиплексирования. Мультиплексирование сообщений. Модель взаимодействия процессов клиент-сервер. Неравноправность клиента и сервера.

Используя технику из предыдущего примера, мы можем организовать получение сообщений одним процессом от множества других процессов через одну очередь сообщений и отправку им ответов через ту же очередь сообщений, т.е. осуществить мультиплексирование сообщений. Вообще под мультиплексированием информации понимают возможность одновременного обмена информацией с несколькими партнерами. Метод мультиплексирования широко применяется в модели взаимодействия процессов клиент-сервер. В этой модели один из процессов является сервером. Сервер получает запросы от других процессов - клиентов - на выполнение некоторых действий и отправляет им результаты обработки запросов. Наиболее часто модель клиент-сервер используется при разработке сетевых приложений. Она изначально предполагает неравноправность взаимодействующих процессов:

Сервер, как правило, работает постоянно, на всем протяжении жизни приложения, а клиенты могут работать эпизодически.

Сервер ждет запроса от клиентов, инициатором же взаимодействия выступает клиент.

Как правило, клиент обращается к одному серверу за раз, в то время как к серверу могут одновременно поступить запросы от нескольких клиентов.

Клиент должен знать, как обратиться к серверу (например, какого типа сообщения он воспринимает) перед началом организации запроса к серверу, в то время как сервер может получить недостающую информацию о клиенте из пришедшего запроса.

Рассмотрим следующую схему мультиплексирования сообщений через одну очередь сообщений для модели клиент-сервер. Пусть сервер получает из очереди сообщений только сообщения с типом 1. В состав сообщений с типом 1, посылаемых серверу, процессы-клиенты включают значение своих идентификаторов процесса. Приняв сообщение с типом 1, сервер анализирует его содержание, выявляет идентификатор процесса, пославшего запрос, и отвечает клиенту, посылая сообщение с типом равным идентификатору запрашивавшего процесса. Процесс-клиент после посылки запроса ожидает ответа в виде сообщения с типом равным своему идентификатору. Поскольку идентификаторы процессов в системе различны, и ни один пользовательский процесс не может иметь PID равный 1, все сообщения могут быть прочитаны только теми процессами, которым они адресованы. Если обработка запроса занимает продолжительное время, сервер может организовывать параллельную обработку запросов, порождая, для каждого запроса, новый процесс-ребенок или новую нить исполнения.

10.Использование очередей сообщений для синхронизации работы процессов.

На лекции была доказана эквивалентность очередей сообщения и семафоров в системах, где процессы могут использовать разделяемую память. Там, в частности, было показано, как реализовать семафоры с помощью очередей сообщений. Для этого вводился специальный синхронизирующий процесс-сервер, обслуживающий переменные-счетчики для каждого семафора. Процессы-клиенты для выполнения операции над семафором посылали процессу-серверу запросы на выполнение операции и ожидали ответа для продолжения работы. Теперь мы знаем, как это можно сделать в операционной системе UNIX и как, следовательно, можно использовать очереди сообщений для организации взаимоисключений и взаимной синхронизации процессов.

Задание:

1. Напишите, откомпилируйте и прогоните программы, осуществляющие двустороннюю связь через одну очередь сообщений.
2. Напишите, откомпилируйте и прогоните программы сервера и клиентов для предложенной схемы мультиплексирования сообщений.
3. Задача повышенной сложности: реализуйте семафоры через очереди сообщений.

Контрольные вопросы

1. Сообщения как средства связи и средства синхронизации процессов.
2. Очереди сообщений в UNIX как составная часть System V IPC.
3. Создание очереди сообщений или доступ к уже существующей. Системный вызов msgget().
4. Реализация примитивов send и receive. Системные вызовы msgsnd() и msgrcv().
5. Удаление очереди сообщений из системы с помощью команды ipcrm или системного вызова msgctl().
6. Понятие мультиплексирования. Мультиплексирование сообщений. Модель взаимодействия процессов клиент-сервер. Неравноправность клиента и сервера.
7. Использование очередей сообщений для синхронизации работы процессов.

ЛАБОРАТОРНЫЕ РАБОТЫ №12. ОРГАНИЗАЦИЯ ВВОДА-ВЫВОДА В UNIX. ФАЙЛЫ УСТРОЙСТВ. АППАРАТ ПРЕРЫВАНИЙ. СИГНАЛЫ В UNIX.

Цели занятия

- Дать понятие об организации управлением ввода-вывода в UNIX.
- Объяснить, что такое файлы устройств и для чего они нужны.
- Объяснить иерархическую систему процессов в UNIX, как процесс может стать лидером группы или лидером сеанса.
- Студент должен получить четкое осознание отличия аппаратных прерываний, исключений и программных прерываний (interrupt, exception, software interrupt или trap).
- Дать понятие об организации сигналов в UNIX.
- Дать навык работы с сигналами от терминала, порожденного процесса, для организации синхронизации процессов.
- Студент должен осознать степень надежности передачи информации процессу с использованием сигналов.

Практические работы

- Изучение особенностей получения терминальных сигналов текущей и фоновой группой процессов.
- Изучение получения сигнала SIGHUP процессами при завершении лидера сеанса.
- Прогон программы, игнорирующей сигнал SIGINT.
- Модификация предыдущей программы для игнорирования сигналов SIGINT и SIGQUIT.
- Прогон программы с пользовательской обработкой сигнала SIGINT.
- Модификация предыдущей программы для пользовательской обработки сигналов SIGINT и SIGQUIT.
- Восстановление предыдущей реакции на сигнал.
- Сигналы SIGUSR1 и SIGUSR2. Использование сигналов для синхронизации процессов.
- Прогон программы для иллюстрации обработки сигнала SIGCHLD.

1.Понятие виртуальной файловой системы.

На предыдущих занятиях мы с вами рассматривали устройство файловой системы s5fs. Существуют и другие файловые системы, имеющие архитектуру, существенно отличающуюся от архитектуры s5fs (иные способы отображения файла на пространство физического носителя, иное построение директорий и т.д.). Современные версии UNIX-подобных операционных систем умеют работать с разнообразными файловыми системами, отличающимися своей организацией. Такая возможность достигается с помощью разбиения каждой существующей файловой системы на зависимую и независимую от конкретной реализации части, подобно тому, как на лекции, посвященной вопросам ввода-вывода, мы отделяли аппаратно-зависимые части для каждого устройства (драйвера) от общей базовой подсистемы ввода-вывода. Независимые части всех файловых систем одинаковы и представляют для всех остальных элементов ядра абстрактную файловую систему, которую принято называть виртуальной файловой системой. Зависимые части для различных файловых систем могут встраиваться в ядро на этапе компиляции, либо добавляться к нему динамически по мере необходимости, без перекомпиляции системы (как в системах с микроядерной архитектурой).

Рассмотрим схематично устройство виртуальной файловой системы. В файловой системе s5fs каждый открытый файл представлялся в операционной системе структурой данных в таблице индексных узлов открытых файлов, содержащей информацию из индексного узла файла во вторичной памяти. В виртуальной файловой системе, в отличие от s5fs, каждый файл характеризуется не индексным узлом inode, а некоторым виртуальным узлом vnode. При

открытии файла в операционной системе для него заполняется (если, конечно, не был заполнен раньше) элемент *таблицы виртуальных узлов* открытых файлов, в котором хранятся, как минимум, тип файла, счетчик числа открытий файла, указатель на реальные физические данные файла и, обязательно, указатель на таблицу системных вызовов, совершающих операции над файлом, - *таблицу операций.* Реальные физические данные файла (равно как и способ расположения файла на диске, и т.п.) и системные вызовы, реально выполняющие операции над файлом, уже не являются элементами виртуальной файловой системы. Они относятся к одной из зависимых частей файловой системы, так как определяются ее конкретной реализацией.

При выполнении операций над файлами по таблице операций, адрес которой содержится в vnode, определяется системный вызов, который будет на самом деле выполнен над реальными физическими данными файла, адрес которых также находится в vnode. В случае с s5fs, данные, на которые ссылается vnode, - это как раз данные индексного узла. Заметим, что таблица операций является общей для всех файлов, принадлежащих одной и той же файловой системе.

2.Операции над файловыми системами. Монтирование файловых систем.

Как только мы переходим к взаимному сосуществованию нескольких файловых систем в рамках одной операционной системы, встает вопрос о логическом объединении структур этих файловых систем. При начале работы операционной системы нам изначально доступна лишь одна, так называемая корневая, файловая система. Прежде, чем приступить к работе с файлом, лежащим в некоторой другой файловой системе, мы должны встроить ее в уже существующий ациклический граф файлов. Эта операция - операция над файловой системой - называется монтированием файловой системы **(mount**).

Для монтирования файловой системы в существующем графе должна быть найдена или создана некоторая пустая директория - точка монтирования, к которой и присоединится корень монтируемой файловой системы. При операции монтировании в ядре заводятся структуры данных, описывающие файловую систему, а в vnode для точки монтирования файловой системы помещается специальная информация.

Монтирование файловых систем обычно является прерогативой системного администратора и осуществляется командой операционной системы mount в ручном режиме, либо автоматически при старте операционной системы. Использование этой команды без параметров не требует специальных полномочий и позволяет пользователю получить информацию обо всех подмонтированных файловых системах и соответствующих им физических устройствах. Для пользователя также обычно разрешается монтирование файловых систем, расположенных на гибких магнитных дисках. Для первого накопителя на гибких магнитных дисках такая команда в Linux будет выглядеть следующим образом:

mount /dev/fd0 <имя пустой директории>,

где ***<имя пустой директории>*** описывает точку монтирования, а ***/dev/fd0*** - специальный файл устройства, соответствующего этому накопителю (о специальных файлах устройств мы подробнее поговорим в следующем разделе).

Если мы не собираемся использовать подмонтированную файловую систему в дальнейшем (например, хотим вынуть ранее подмонтированную дискету), нам необходимо выполнить операцию логического разъединения смонтированных файловых систем (umount). Для этой операции, которая тоже, как правило, является привилегией системного администратора, используется команда umount (может выполняться в ручном режиме или автоматически при завершении работы операционной системы). Для пользователя обычно доступна команда по отмонтированию файловой системы на дискете в форме

umount <имя точки монтирования>,

где ***<имя точки монтирования>*** - это ***<имя пустой директории>***, использованное ранее в команде mount, или в форме

umount /dev/fd0,

где ***/dev/fd0*** - специальный файл устройства, соответствующего первому накопителю на гибких магнитных дисках. Заметим, что для последующей корректной работы операционной системы при удалении физического носителя информации, обязательно необходимо предварительное логическое разъединение файловых систем, если они перед этим были объединены.

3.Блочные, символьные устройства. Понятие драйвера. Блочные, символьные драйверы, драйверы низкого уровня. Файловый интерфейс.

Обремененные знаниями об устройстве современных файловых систем в UNIX, мы можем, наконец, заняться вопросами реализации подсистемы ввода-вывода.

На лекции мы говорили о том, что все возможные устройства ввода-вывода можно разделить на относительно небольшое число типов, отличающихся по набору операций, которые могут быть ими выполнены, считая все остальные различия несущественными. Такое деление позволяет организовать подсистему ввода-вывода слоеным образом, вынеся все аппаратно-зависимые части в драйвера устройств, с которыми взаимодействует базовая подсистема ввода-вывода, осуществляющая стратегическое управление всеми устройствами.

В операционной системе UNIX принята упрощенная классификация устройств: все устройства разделяются по способу передачи данных на символьные и блочные. Символьные устройства осуществляют передачу данных байт за байтом, в то время как блочные устройства передают блок байт как единое целое. Типичным примером символьного устройства является клавиатура, типичным примером блочного устройства является жесткий диск. Непосредственное взаимодействие операционной системы с устройствами ввода-вывода обеспечивают их драйверы. Существует пять основных случаев, когда ядро обращается к драйверам:

- Автоконфигурация. Происходит в процессе инициализации операционной системы, когда ядро определяет наличие доступных устройств.
- Ввод-вывод. Обработка запроса ввода-вывода.
- Обработка прерываний. Ядро вызывает специальные функции драйвера для обработки прерывания, поступившего от устройства, в том числе, возможно, для планирования очередности запросов к нему.
- Специальные запросы. Например, изменение параметров драйвера или устройства.
- Повторная инициализация устройства или останов операционной системы.

Так же как устройства подразделяются на символьные и блочные, так и драйверы существуют символьные и блочные. Особенностью блочных устройств является возможность организации на них файловой системы, поэтому блочные драйверы обычно используются файловой системой UNIX. При обращении к блочному устройству, не содержащему файловой системы, применяются специальные драйверы низкого уровня, как правило, представляющие собой интерфейс между ядром операционной системы и блочным драйвером устройства.

Для каждого из этих трех типов драйверов были выделены основные функции, которые базовая подсистема ввода-вывода может совершать над устройствами и драйверами: инициализация устройства или драйвера, временное завершение работы устройства, чтение, запись, обработка прерывания, опрос устройства и т.д. (об этих операциях мы уже говорили на лекции). Эти функции были систематизированы и представляют собой интерфейс между драйверами и базовой подсистемой ввода-вывода.

Каждый драйвер определенного типа в операционной системе UNIX получает свой собственный номер, который по сути дела является индексом в массиве специальных структур данных операционной системы - коммутаторе устройств соответствующего типа. Этот индекс принято также называть *старшим номером устройства*, хотя на самом деле он относится не к устройству, а к драйверу. Несмотря на наличие трех типов драйверов, в операционной системе используется всего два коммутатора: для блочных и символьных драйверов. Драйвера низкого уровня распределяются между ними по преобладающему типу интерфейса (к какому типу ближе - в такой массив и заносятся). Каждый элемент коммутатора устройств обязательно

содержит адреса (точки входа в драйвер), соответствующие стандартному набору функций интерфейса, которые и вызываются операционной системой для выполнения тех или иных действий над устройством и/или драйвером.

Помимо старшего номера устройства существует еще и *младший номер устройства,* который передается драйверу в качестве параметра и смысл которого определяется самим драйвером. Например, это может быть номер раздела на жестком диске (partition), доступ к которому должен обеспечить драйвер (надо отметить, что в операционной системе UNIX различные разделы физического носителя информации рассматриваются как различные устройства). В некоторых случаях младший номер устройства может не использоваться, но для единообразия он обязан присутствовать. Таким образом, пара драйвер-устройство всегда однозначно определяется в операционной системе заданием пары номеров (старшего и младшего номеров устройства) и типа драйвера (символьный или блочный).

Для связи приложений с драйверами устройств операционная система UNIX использует *файловый интерфейс*. Каждой тройке тип-драйвер-устройство в файловой системе соответствует специальный файл устройства, который не занимает на диске никаких логических блоков, кроме индексного узла. В качестве атрибутов этого файла помимо обычных атрибутов используются соответствующие старший и младший номера устройства и тип драйвера (тип драйвера определяется по типу файла - ибо есть специальные файлы символьных устройств и специальные файлы блочных устройств, а номера устройств занимают место длины файла для, скажем, регулярных файлов). Когда открывается специальный файл устройства, операционная система, в числе прочих действий, заносит в соответствующий элемент таблицы открытых виртуальных узлов указатель на набор функций интерфейса из соответствующего элемента коммутатора устройств. Теперь при попытке чтения из файла устройства или записи в файл устройства виртуальная файловая система будет транслировать запросы на выполнение этих операций в необходимые вызовы нужного драйвера.

Мы не будем останавливаться на практическом применении файлового интерфейса для работы с устройствами ввода-вывода, поскольку это выходит за пределы нашего курса, а вместо этого приступим к изложению концепции сигналов в UNIX, тесно связанных с понятиями аппаратного прерывания, исключения и программного прерывания.

4.Аппаратные прерывания (interrupt), исключения (exception), программные прерывания (trap, software interrupt). Их обработка.

На лекциях уже вводились понятия аппаратного прерывания, исключения и программного прерывания. Кратко напомним сказанное.

После выдачи запроса ввода-вывода у процессора существует два способа узнать о том, что обработка запроса устройством завершена. Первый способ заключается в регулярной проверке процессором ***бита занятости*** в ***регистре состояния*** контроллера соответствующего устройства (polling). Второй способ заключается в использовании прерываний. При втором способе процессор имеет специальный вход, на который устройства ввода-вывода непосредственно, или используя контроллер прерываний, выставляют сигнал запроса прерывания (interrupt request) при завершении операции ввода-вывода. При наличии такого сигнала, процессор, после выполнения текущей команды, не выполняет следующую, а, сохранив состояние ряда регистров и, возможно, загрузив в часть регистров новые значения, переходит на выполнение команд, расположенных по некотором фиксированным адресам. После окончания обработки прерывания можно восстановить состояние процессора и продолжить его работу с команды, выполнение которой было отложено.

Аналогичный механизм часто используется при обработке исключительных ситуаций (exception), возникающих при выполнении команды процессором (неправильный адрес в команде, защита памяти, возникло деление на ноль и т.д.). В этом случае процессор не заканчивает выполнение команды, а поступает, как и при прерывании, сохраняя свое состояние до момента начала ее выполнения.

Этим же механизмом часто пользуются и для реализации так называемых программных прерываний (software interrupt, trap), используемых, например, для переключения процессора из режима пользователя в режим ядра внутри системных вызовов. Для выполнения действий аналогичных действиям по обработке прерывания процессор, в этом случае, должен выполнить специальную команду.

Необходимо четко представлять себе разницу между этими тремя понятиями, для чего не лишним будет в очередной раз обратиться к лекциям.

Как правило, обработку аппаратных прерываний от устройств ввода-вывода производит сама операционная система, не доверяя работу с системными ресурсами процессам пользователя. Обработка же исключительных ситуаций и некоторых программных прерываний вполне может быть возложена на пользовательский процесс через механизм сигналов.

5.Понятие сигнала. Способы возникновения сигналов и виды их обработки.

С точки зрения пользователя получение процессом сигнала выглядит как возникновение прерывания. Процесс прекращает свое регулярное исполнение, и управление передается механизму обработки сигнала. По окончании обработки сигнала процесс может возобновить регулярное исполнение. Типы сигналов (их принято задавать номерами, как правило, в диапазоне от 1 до 31 включительно или специальными символьными обозначениями) и способы их возникновения в системе жестко регламентированы.

Процесс может получить сигнал от:

- Hardware (при возникновении исключительной ситуации)
- Другого процесса, выполнившего системный вызов передачи сигнала.
- Операционной системы (при наступлении некоторых событий).
- Терминала (при нажатии определенной комбинации клавиш).
- Системы управления заданиями (при выполнении команды **kill** - мы рассмотрим ее позже).

Передачу сигналов процессу в случаях его генерации источниками 2, 3 и 5, т.е. в конечном счете, каким-либо другим процессом, можно рассматривать как реализацию в UNIX сигнальных средств связи.

Существует три варианта реакции процесса на сигнал:

1. Принудительно проигнорировать сигнал.
2. Произвести обработку по умолчанию (проигнорировать, остановить процесс (перевести в состояние ожидания до получения другого специального сигнала), либо завершить работу с образование core файла или без оного).
3. Выполнить обработку сигнала, специфицированную пользователем.

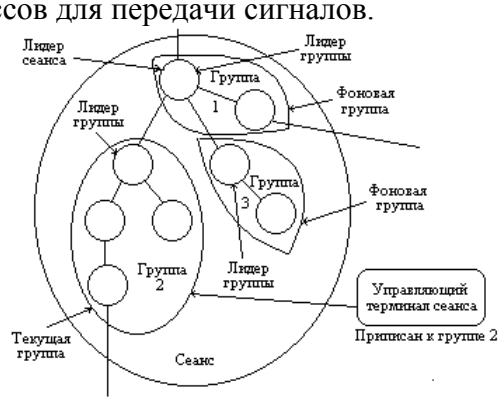
Изменить реакцию процесса на сигнал можно с помощью специальных системных вызовов, которые мы рассмотрим позже. Реакция на некоторы сигналы не допускает изменения, и они могут быть обработаны только по умолчанию. Так, например, сигнал с номером 9 - **SIGKILL** обрабатывается только по умолчанию и всегда приводит к завершению процесса.

Важным вопросом при программировании с использованием сигналов является вопрос о сохранении реакции на них при рождении нового процесса или замене его пользовательского контекста. При системном вызове fork() все установленные реакции на сигналы наследуется порожденным процессом. При системном вызове exec() сохраняются реакции только для тех сигналов, которые игнорировались или обрабатывались по умолчанию. Получение любого сигнала, который до вызова exec() обрабатывался пользователем, приведет к завершению процесса.

6.Понятия группы процессов, сеанса, лидера группы, лидера сеанса, управляющего терминала сеанса. Системные вызовы getpgrp(), setpgrp(), getpgid(), setpgid(), getsid(), setsid()

Мы уже говорили, что все процессы в системе связаны родственными отношениями, образуя генеалогическое дерево или лес из таких деревьев, где в качестве узлов деревьев выступают сами процессы, а связями служат отношения родитель-ребенок. Все эти деревья принято разделять на группы процессов, так сказать, семьи (см. рисунок).

Группа процессов включает в себя один или более процессов и существует, пока в группе присутствует хотя бы один процесс. Каждый процесс обязательно включен в какую-нибудь группу. При рождении нового процесса он попадает в ту же группу процессов, в которой находится его родитель. Процессы могут мигрировать из группы в группу по своему собственному желанию или по желанию другого процесса (в зависимости от версии UNIX). Многие системные вызовы могут быть применены не к одному конкретному процессу, а ко всем процессам в некоторой группе. Поэтому то, как именно вы будете объединять процессы в группы, зависит от того, как вы собираетесь их использовать. Чуть позже мы поговорим об использовании групп процессов для передачи сигналов.



В свою очередь, группы процессов объединяются в сеансы, образуя с родственной точки зрения некие кланы семей. Понятие сеанса изначально было введено в UNIX для логического объединения групп процессов, созданных в результате каждого входа и последующей работы пользователя в системе. С каждым сеансом, поэтому, может быть связан в системе терминал, называемый управляющим терминалом сеанса, через который обычно и общаются процессы сеанса с пользователем. Сеанс не может иметь более одного управляющего терминала, и один терминал не может быть управляющим для нескольких сеансов. В то же время могут существовать сеансы, вообще не имеющие управляющего терминала.

Каждая группа процессов в системе получает свой собственный уникальный номер. Узнать этот номер можно с помощью системного вызова getpgid(). Используя его, процесс может узнать номер группы для себя самого или для процесса из своего сеанса. К сожалению, не во всех версиях UNIX присутствует данный системный вызов. Здесь мы сталкиваемся с тяжелым наследием разделения линий UNIX'ов на линию BSD и линию System V, которое будет нас преследовать почти на всем протяжении данной темы. Вместо вызова getpgid() в таких системах существует системный вызов getpgrp(), который возвращает вам номер группы только для текущего процесса.

Для перевода процесса в другую группу процессов, возможно с одновременным ее созданием, применяется системный вызов setpgid(). Перевести в другую группу процесс может либо самого себя (и то не во всякую и не всегда), либо свой процесс-ребенок, который не выполнял системный вызов exec(), т.е. не запускал на выполнение другую программу. При определенных значениях параметров системного вызова создается новая группа процессов с идентификатором, совпадающим с идентификатором переводимого процесса, состоящая первоначально только из одного этого процесса. Новая группа может быть создана только

таким способом, поэтому идентификаторы групп в системе уникальны. Переход в другую группу без создания новой группы возможен только в пределах одного сеанса.

В некоторых разновидностях UNIX системный вызов setpgid() отсутствует, а вместо него существует системный вызов setpgrp(), который умеет только создавать новую группу процессов с идентификатором, совпадающим с идентификатором текущего процесса, и переводить текущий процесс в нее. (В некоторых разновидностях UNIX, где совместно сосуществуют вызовы setpgrp() и setpgid(), например в Solaris, вызов setpgrp() ведет себя иначе - он аналогичен рассматриваемому ниже вызову setsid().)

Процесс, идентификатор которого совпадает с идентификатором его группы, называется лидером группы. Одно из ограничений на применение вызовов setpgid() и setpgrp() состоит в том, что лидер группы не может перебраться в другую группу.

Каждый сеанс в системе также имеет свой собственный номер. Для того, чтобы узнать его можно воспользоваться системным вызовом getsid(). В разных версиях UNIX на него накладываются различные ограничения. В Linux такие ограничения отсутствуют.

Использование системного вызова setsid() приводит к созданию новой группы, состоящий только из процесса, который его выполнил, (он становится лидером новой группы), и нового сеанса, идентификатор которого совпадает с идентификатором процесса, сделавшего вызов. Такой процесс называется лидером сеанса. Этот системный вызов может применять только процесс, не являющийся *лидером группы*.

Если сеанс имеет управляющий терминал, то он обязательно приписывается к некоторой группе процессов, входящей в сеанс. Такая группа процессов называется *текущей группой процессов* для данного сеанса. Все процессы, входящие в текущую группу процессов могут совершать операции ввода-вывода, используя управляющий терминал. Все остальные группы процессов сеанса называются фоновыми группами, а процессы, входящие в них - фоновыми процессами. При попытке ввода-вывода *фонового процесса* через управляющий терминал, этот процесс получит сигналы, которые стандартно приводят к прекращению работы процесса. Передавать управляющий терминал от одной группы процессов к другой может только лидер сеанса. Заметим, что для сеансов, не имеющих управляющего терминала, все процессы являются фоновыми.

При завершении работы процесса - лидера сеанса все процессы из текущей группы сеанса получают сигнал **SIGHUP**, который при стандартной обработке приведет к их завершению. Таким образом, после завершения лидера сеанса в нормальной ситуации работать продолжат только фоновые процессы.

Процессы, входящие в текущую группу сеанса, могут получать сигналы, инициируемые нажатием определенных клавиш на терминале - **SIGINT** при нажатии клавиш <ctrl> и <c>, и SIGQUIT при нажатии клавиш <ctrl> и <4>. Стандартная реакция процесса на эти сигналы - завершение процесса.

Нам понадобится еще одно понятие, связанное с процессом, - эффективный идентификатор пользователя. В начале семестра мы говорили, что каждый пользователь в системе имеет свой собственный идентификатор - **UID**. Каждый процесс, запущенный пользователем, использует этот **UID** для определения своих полномочий. Однако иногда, если у исполняемого файла были выставлены соответствующие атрибуты, процесс может прикинуться процессом, запущенным другим пользователем. Идентификатор пользователя, от имени которого процесс пользуется полномочиями, и является эффективным идентификатором пользователя для процесса - **EUID**. За исключением выше оговоренного случая, эффективный идентификатор пользователя совпадает с идентификатором пользователя, создавшего процесс.

7.Системный вызов kill() и команда kill().

Из всех перечисленных ранее источников сигнала пользователю доступны только два - команда kill и посылка сигнала процессу с помощью системного вызова kill(). Команда kill обычно используется в следующей форме:

kill [-номер] pid

Здесь **pid** - это идентификатор процесса, которому посылается сигнал, а "**номер**"- номер сигнала, который посылается процессу. Послать сигнал (если у вас нет полномочий суперпользователя) можно только процессу, у которого эффективный идентификатор пользователя совпадает с идентификатором пользователя, посылающего сигнал. Если параметр **-номер** отсутствует, то посылается сигнал **SIGTERM**, обычно имеющий номер **15** и реакция на него по умолчанию - завершить работу процесса, который получил сигнал.

При использовании системного вызова kill() послать сигнал (если у вас нет полномочий суперпользователя) можно только процессу или процессам, у которых эффективный идентификатор пользователя совпадает с эффективным идентификатором пользователя процесса, посылающего сигнал.

8.Изучение особенностей получения терминальных сигналов текущей и фоновой группой процессов.

Давайте возьмем тривиальную программу \\ИБ3\БОС\linux\13-1.c, в которой процесс порождает ребенка, и они оба зацикливаются, и на ее основе проиллюстрируем сказанное выше.

Для этого мы будем использовать команду **ps** с опцией **-ej**, которая позволяет получить информацию о всех процессах в системе и узнать их идентификаторы, идентификаторы групп процессов и сеансов, управляющий терминал сеанса и к какой группе процессов он приписан. Набрав команду "**ps -ej**" мы получим список всех процессов в системе. Колонка **PID** содержит идентификаторы процессов, колонка **PGID** - идентификаторы групп, к которым они принадлежат, колонка **SID** - идентификаторы сеансов, колонка **TTY** - номер соответствующего управляющего терминала, колонка **TPGID** (может присутствовать не для всех версий UNIX, но в Linux есть) - к какой группе процессов приписан управляющий терминал.

Откомпилируйте тривиальную программу и запустите ее на исполнение (лучше всего из под оболочки Midnight Commander - mc). Запустив команду "**ps -ej**" с другого экрана, проанализируйте значения идентификаторов группы процессов, сеансов, прикрепления управляющего терминала, текущей и фоновой групп. Убедитесь, что тривиальные процессы относятся к текущей группе сеанса. Проверьте реакцию текущей группы на сигналы **SIGINT** - клавиши <CTRL> и <C> - и сигнал **SIGQUIT -** клавиши <CTRL> и <4>.

Запустите теперь тривиальную программу в фоновом режиме, например командой "oa.out &". Проанализируйте значения идентификаторов группы процессов, сеансов, прикрепления управляющего терминала, текущей и фоновой групп. Убедитесь, что тривиальные процессы относятся к фоновой группе сеанса. Проверьте реакцию фоновой группы на сигналы **SIGINT** - клавиши <CTRL> и <C> - и сигнал **SIGQUIT** - клавиши <CTRL> и <4>. Ликвидируйте тривиальные процессы с помощью команды kill.

9.Системный вызов signal(). Установка собственного обработчика сигнала.

Одним из способов изменения поведения процесса при получении сигнала в операционной системе UNIX является использование системного вызова signal().

Этот системный вызов имеет два параметра: один из них задает номер сигнала, реакцию процесса на который мы хотим изменить, а второй определяет, как именно мы собираемся ее менять. Для первого варианта реакции процесса на сигнал - его игнорирования - применится специальное значение этого параметра **SIG_IGN**. Например, если требуется игнорировать сигнал **SIGINT**, начиная с некоторого места работы программы, в этом месте программы мы должны употребить конструкцию

(void) signal(SIGINT, SIG_IGN);

Для второго варианта реакции процесса на сигнал - восстановления его обработки по умолчанию - применится специальное значение этого параметра **SIG_DFL**. Для третьего

варианта реакции процесса на сигнал в качестве значения параметра подставляется указатель на пользовательскую функцию обработки сигнала, которая должна иметь прототип вида

void *handler(int);

Ниже приведен пример скелета конструкции для пользовательской обработки сигнала **SIGHUP**.

```
void *my_handler(int nsig) {
<обработка сигнала>
}
int main() {
...
(void)signal(SIGHUP, my_handler);
...
}
```

В качестве значения параметра в пользовательскую функцию обработки сигнала (в нашем склете - параметр nsig) передается номер возникшего сигнала, так что одна и та же функция может быть использована для различной обработки нескольких сигналов.

10.Прогон программы, игнорирующей сигнал SIGINT.

Давайте рассмотрим программу, расположенную в файле \\ИБ3\БОС\linux\/13-3.c. Эта программа не делает ничего полезного, кроме переустановки реакции на нажатие клавиш <ctrl> и <c> на игнорирование возникающего сигнала и своего бесконечного зацикливания.

11.Восстановление предыдущей реакции на сигнал.

До сиз пор в примерах мы игнорировали значение, возвращаемое системным вызовом signal(). На самом деле этот системный вызов возвращает указатель на предыдущий обработчик сигнала, что позволяет восстанавливать переопределенную реакцию на сигнал.

12.Сигналы SIGUSR1 и SIGUSR2. Использование сигналов для синхронизации процессов.

В операционной системе UNIX существует два сигнала, источниками которых могут служить только системный вызов kill() или команда kill, - это сигналы **SIGUSR1** и **SIGUSR2.** Обычно их применяют для передачи информации о происшедшем событии от одного пользовательского процесса другому в качестве сигнального средства связи между процессами пользователя.

Организуйте двусторонную поочередную связь процесса-родителя и процесса-ребенка через pipe, используя для синхронизации сигналы **SIGUSR1** и **SIGUSR2**, модифицировав программу \\ИБ3\БОС\linux\05-3.c.

Задача повышенной сложности: организуйте побитовую передачу целого числа между двумя процессами, используя для этого только сигналы **SIGUSR1** и **SIGUSR2**.

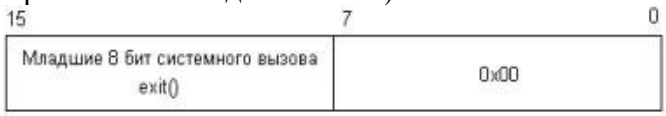
При реализации нитей исполнения в операционной системе Linux сигналы **SIGUSR1** и **SIGUSR2** используются для организации синхронизации между процессами, представляющими нити исполнения, и процессом-координатором в служебных целях. Поэтому пользовательские программы, использующие в своей работе нити исполнения не могут использовать сигналы **SIGUSR1** и **SIGUSR2**.

13.Завершение порожденного процесса. Системный вызов waitpid(). Сигнал SIGCHLD.

При изучении завершения процесса, мы с вами говорили о том, что если процесс-ребенок завершает свою работу прежде процесса-родителя, и процесс-родитель явно не указал, что он не заинтересован в получении информации о статусе завершения процесса-ребенка, то завершившийся процесс не исчезает из системы окончательно, а остается в состоянии ***закончил исполнение*** (зомби-процесс) либо до завершения процесса-родителя, либо до того момента, когда родитель соблаговолит получить эту информацию.

Для получения такой информации процесс-родитель может воспользоваться системным вызовом waitpid() или его упрощенной формой wait(). Системный вызов waitpid() позволяет процессу-родителю синхронно получить данные о статусе завершившегося процесса-ребенка, либо блокируя процесс-родитель до завершения процесса-ребенка, либо без блокировки при его периодическом вызове с опцией **WNOHANG**. Эти данные занимают 16-бит и, в рамках нашего курса, могут быть расшифрованы следующим образом:

Если процесс завершился при помощи явного или неявного вызова функции exit(), то данные выглядят так (старший бит находится слева)



Если процесс был завершен сигналом, то данные выглядят так (старший бит находится слева)



Каждый процесс-ребенок при завершении работы посылает своему процессу-родителю специальный сигнал **SIGCHLD,** на который у всех процессов по умолчанию установлена реакция "игнорировать сигнал". Наличие такого сигнала совместно с системным вызовом waitpid() позволяет организовать асинхронный сбор информации о статусе завершившихся порожденных процессов процессом-родителем.

Используя системный вызов signal() мы можем явно установить игнорирование этого сигнала **(SIG_IGN)**, тем самым, проинформировав систему, что нас не интересует, каким образом завершатся порожденные процессы. В этом случае зомби-процессов возникать не будет, но и применение системных вызовов wait() и waitpid() будет запрещено.

14.Прогон программы для иллюстрации обработки сигнала SIGCHLD.

Для закрепления сказанного давайте рассмотрим пример программы с асинхронным получением информации о статусе завершения порожденного процесса - \\ИБ3\БОС\linux\/stud/s13-6.c. В этой программе родитель порождает 2 процесса. Один из них завершается с кодом 200, а второй - циклится. Перед порождением процессов родитель устанавливает обработчик прерывания для сигнала **SIGCHLD,** а после их порождения уходит в бесконечный цикл. В обработчике прерывания вызывается waitpid() для любого порожденного процесса. Так как в обработчик мы попадаем, когда какой-либо из процессов завершился, то системный вызов не блокируется, и мы можем получить информацию об идентификаторе завершившегося процесса и причине его завершения. Откомпилируйте программу и запустите ее на исполнение. Второй порожденный процесс завершайте с помощью команды kill с каким-либо номером сигнала. Родительский процесс также будет необходимо завершать командой kill.

15.Возникновение сигнала SIGPIPE при попытке записи в pipe или FIFO, который никто не собирается читать.

Как говорилось выше системные вызовы read() и write() имеют определенные особенности поведения. Одной из таких особенностей является получение сигнала SIGPIPE процессом, который пытается записывать информацию в pipe или в FIFO в том случае, когда читать ее уже некому (нет ни одного процесса, который имеет соответствующий pipe или FIFO открытым для чтения). Реакция по умолчанию на этот сигнал - прекратить работу процесса. Теперь мы с вами уже можем написать корректную обработку этого сигнала пользователем для, например, элегантного прекращения работы пишущего процесса. Однако, нам для полноты картины

необходимо познакомиться с особенностями поведения некоторых системных вызовов при получении сигналов процессом во время их выполнения.

По ходу нашего курса мы с вами познакомились с рядом системных вызовов, которые могут блокировать процесс во время своего выполнения. К их числу относятся системный вызов open() при открытии FIFO, системные вызовы read() и write() при работе с pip'ами и FIFO, системные вызовы msgsnd() и msgrcv() при работе с очередями сообщений, системный вызов semop() при работе с семафорами и т.д. Что произойдет с процессом, если он, выполняя один из этих системных вызовов, получит какой-либо сигнал. Дальнейшее поведение процесса зависит от установленной для него реакции на этот сигнал.

Если реакция на полученный сигнал была "игнорировать сигнал" (независимо от того, установлена она по умолчанию или пользователем с помощью системного вызова signal()), то поведение процесса не изменится.

Если реакция на полученный сигнал установлена по умолчанию и заключается в прекращении работы процесса, то процесс перейдет в состояние ***закончил исполнение***.

Если реакция процесса на сигнал заключается в выполнении пользовательской функции, то процесс выполнит эту функцию (если он находился в состоянии ожидание, он перейдет в состояние ***готовность*** и затем в состояние ***исполнение***) и вернется из системного вызова с констатацией ошибочной ситуации. Отличить такой возврат от действительно ошибочной ситуации можно с помощью значения системной переменной **errno**, которая в этом случае примет значение **EINTR**.

После этого краткого обсуждения становится до конца ясно, как корректно обработать ситуацию "никто не хотел прочитать" для системного вызова write(). Чтобы пришедший сигнал **SIGPIPE** не завершил работу нашего процесса по умолчанию, мы должны его самостоятельно обработать (функция-обработчик при этом может быть и пустой!). Но этого мало. Поскольку нормальный ход выполнения системного вызова был нарушен сигналом, мы вернемся из него с отрицательным значением, которое свидетельствует об ошибке. Проанализировав значение системной переменной **errno** на предмет совпадения со значением **EINTR,** мы можем отличить возникновение сигнала **SIGPIPE** от других ошибочных ситуаций (неправильные значения параметров и т.д.) и грациозно продолжить работу программы.

Задание:

1. Возьмите снова тривиальную программу и запустите ее на исполнение из под Midnight Commander в текущей группе. Проанализировав значения идентификаторов группы процессов, сеансов, прикрепления управляющего терминала, текущей и фоновой групп, убейте лидера сеанса для тривиальных процессов. Убедитесь, что все процессы в текущей группе этого сеанса прекратили свою работу.
2. Запустите тривиальную программу в фоновом режиме. Снова убейте лидера сеанса для тривиальных процессов. Убедитесь, что фоновая группа продолжает работать. Ликвидируйте тривиальные процессы.
3. Откомпилируйте и запустите программу, расположенную в файле \\ИБ3\БОС\linux\/13-3.c.. Убедитесь, что на нажатие клавиш <ctrl> и <c> она не реагирует, а реакция на нажатие клавиш <ctrl> и <4> осталась прежней.
4. Модифицируйте программу из предыдущего примера так, чтобы она перестала реагировать и на нажатие клавиш <ctrl> и <4>. Откомпилируйте и запустите ее, убедитесь в отсутствии ее реакций на внешние раздражители. Снимать программу придется теперь с другого терминала командой kill.
5. Давайте рассмотрим теперь программу, расположенную в файле \\ИБ3\БОС\linux\/13-4.c. Эта программа отличается от программы из пункта 11 тем, что в ней введена обработка сигнала **SIGINT** пользовательской функцией. Откомпилируйте и запустите эту программу, проверьте ее реакцию на нажатие клавиш <ctrl> и <c> и на нажатие клавиш <ctrl> и <4> .

6. Модифицируйте программу из предыдущего примера так, чтобы она печатала сообщение и о нажатии клавиш <ctrl> и <4>. Используйте одну и ту же функцию для обработки сигналов **SIGINT** и **SIGQUIT**. Откомпилируйте и запустите ее, убедитесь в правильной работе. Снимать программу также придется с другого терминала командой kill.
7. Рассмотрите пример программы, возвращающей первоначальную реакцию на сигнал **SIGINT** после 5 пользовательских обработок сигнала, - \\ИБ3\БОС\linux\/stud/13-5.c, откомпилируйте ее и запустите на исполнение (пункт 11).
8. Задача повышенной сложности: Модифицируйте обработку сигнала в программе \\ИБ3\ БОС\linux\/stud/13-7.c, не применяя POSIX сигналы, так, чтобы процесс-родитель все-таки сообщал о статусе всех завершившихся детей.

Контрольные вопросы

1. Понятие виртуальной файловой системы.
2. Операции над файловыми системами. Монтирование файловых систем.
3. Блочные, символьные устройства. Понятие драйвера. Блочные, символьные драйверы, драйверы низкого уровня. Файловый интерфейс.
4. Аппаратные прерывания (interrupt), исключения (exception), программные прерывания (trap, software interrupt). Их обработка.
5. Понятие сигнала. Способы возникновения сигналов и виды их обработки.
6. Понятия группы процессов, сеанса, лидера группы, лидера сеанса, управляющего терминала сеанса. Системные вызовы getpgrp(), setpgrp(), getpgid(), setpgid(), getsid(), setsid()
7. Системный вызов kill() и команда kill().
8. Системный вызов signal(). Установка собственного обработчика сигнала.
9. Восстановление предыдущей реакции на сигнал.
10. Сигналы SIGUSR1 и SIGUSR2. Использование сигналов для синхронизации процессов.
11. Завершение порожденного процесса. Системный вызов waitpid(). Сигнал SIGCHLD.
12. Возникновение сигнала SIGPIPE при попытке записи в pipe или FIFO, который никто не собирается читать.
13. Понятие о надежности сигналов. POSIX функции для работы с сигналами.